



**Hochschule für Technik
und Wirtschaft Berlin**

University of Applied Sciences

From Feature Importance to Causal Effects: A SHAP and Double Machine Learning Analysis of Time Series in Renewable Energy

Master's Thesis

Name of the Study Program

Finanzmathematik, Aktuarwissenschaften und
Risikomanagement

Faculty 4

submitted by

Duc Tung Bui

Date:

Berlin, April 02, 2026

1. Examiner: Prof. Dr. Christina Erlwein-Sayer
2. Examiner: Prof. Dr. Sunna Torge

Contents

1	Introduction	3
2	Extreme Gradient Boosting (XGBoost)	4
2.1	Fundamental Tree-Based Algorithms	4
2.1.1	Decision Trees	4
2.1.2	Ensemble Learning	4
2.1.3	Gradient Boosting	5
2.2	Definition and Methodology of XGBoost	6
3	Post Hoc Explainability	11
3.1	Shapley Additive Explanations (SHAP)	11
3.2	KernelSHAP	14
3.3	TreeSHAP	16
4	Causal Inference Framework	18
4.1	Directed Acyclic Graphs (DAG)	19
4.2	Average Treatment Effect (ATE)	21
4.3	Propensity Score	22
5	Double Machine Learning	24
5.1	Challenges in High-Dimensional Causal Inference	24
5.2	Target Parameters and Nuisance Functions	24
5.3	Regularization Bias and Overfitting Bias	26
5.4	Motivation for Double Machine Learning	27
5.5	Neyman Orthogonality	28
5.6	Cross-Fitting	29
5.7	The DML Estimator and Estimation Procedure	30
5.8	Interpretation and Limitations	32
6	Double Machine Learning for Time-Series Data with Neighbors-Left-Out Cross-Fitting	34
7	Empirical Application	39
7.1	Dataset Description	39
7.2	Data Preprocessing	40
7.3	Exploratory Data Analysis	43
7.4	Empirical Results	47
7.4.1	XGBoost Model Specification and Predictive Performance	47
7.4.2	SHAP-Based Interpretation of the XGBoost Model	48
7.4.3	NLO-DML Estimate for Solar Generation	51
7.4.4	NLO-DML Estimate for Wind Generation	52
8	Discussion	55
9	Extensions and Future Research	57
10	Conclusion	58
	Literature	59

List of Figures	62
List of Abbreviations	63

1 Introduction

Global electricity demand is increasing because of population growth, economic development, digitalisation, and the expansion of energy-intensive activities such as data centres. This development raises the question of how growing electricity demand can be met while electricity prices remain affordable and stable. Traditional electricity generation based on fossil fuels continues to play an important role in the energy system, but it has harmful environmental effects through greenhouse-gas emissions, air pollution, and its contribution to climate change. Renewable energy sources, particularly solar and wind power, therefore represent a crucial alternative to conventional fossil-fuel-based generation and can substantially reduce greenhouse-gas emissions¹.

However, the expansion of renewable generation requires considerable investment in technologies such as photovoltaic systems, wind turbines, grid connections, and related infrastructure. This raises the question of whether increasing renewable generation contributes not only to environmental protection but also to lower electricity prices. This thesis therefore investigates the effects of actual solar and wind generation on German day-ahead electricity prices, with both renewable energy sources analysed separately.

The empirical analysis applies Double Machine Learning (DML)², a modern causal-inference framework that incorporates machine learning, to estimate the effects of solar and wind generation on German day-ahead electricity prices. The analysis uses hourly time-series data on electricity prices, electricity demand, renewable generation, and installed renewable capacity. XGBoost is used to predict electricity prices and to estimate the nuisance functions required by DML, while SHAP is applied separately to explain the XGBoost price predictions. Because the observations are temporally dependent, Neighbors-Left-Out (NLO) cross-fitting is used to adapt the standard DML procedure to the time-series setting³.

The thesis is structured as follows. First, the machine-learning method XGBoost and its methodological foundations are introduced. This is followed by SHAP and related post-hoc explainability methods used to interpret the XGBoost predictions. The thesis then presents the causal-inference framework and the main methodological approach, Double Machine Learning. Subsequently, the standard DML framework is adapted to time-series data through Neighbors-Left-Out cross-fitting. Finally, the empirical application describes the dataset, data preprocessing, exploratory analysis, and the predictive and causal results for solar and wind generation.

¹Chowdhury and Das 2024.

²Chernozhukov et al. 2018.

³Semenova et al. 2023.

2 Extreme Gradient Boosting (XGBoost)

Before introducing the main concepts and methodology of XGBoost, several fundamental tree-based algorithms are first presented to provide the necessary background for a better understanding of the model.

2.1 Fundamental Tree-Based Algorithms

2.1.1 Decision Trees

A decision tree is a supervised learning model that can be used for both classification and regression tasks, depending on the type of target variable. It predicts an outcome by recursively splitting the data into smaller subsets according to decision rules based on the input features. Although decision trees have many variants, their basic structure generally consists of a root node, internal decision nodes, branches, and terminal leaf nodes, resembling the structure of a tree. In general, the target variable may have different levels of measurement, such as nominal, ordinal, or interval scales⁴. Each internal node represents a split based on a feature, each branch represents the outcome of that split, and each leaf node provides the final prediction. Decision trees are often considered interpretable because they decompose a complex prediction problem into a sequence of smaller decision rules. In this way, each path from the root node to a leaf node represents a sequence of decisions that leads to a particular prediction⁵.

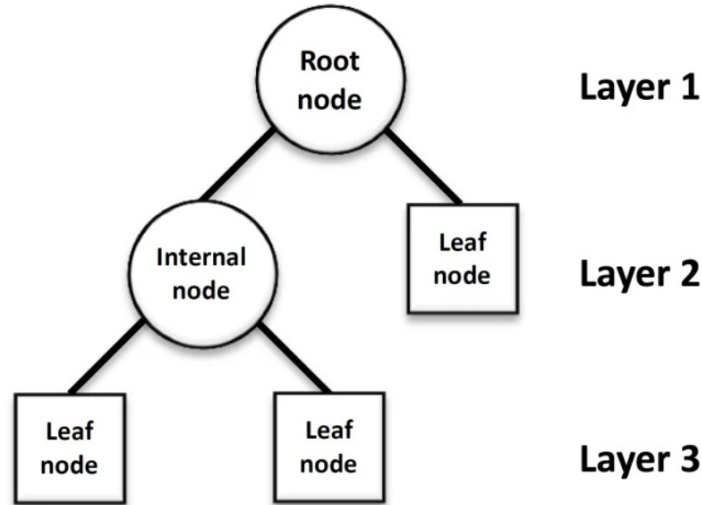


Figure 1: Basic structure of a decision tree. ⁶

2.1.2 Ensemble Learning

Ensemble learning is a machine learning approach in which multiple models, often referred to as base learners, are trained to solve the same prediction problem. Individual learners do not need to be perfect, because their predictions are combined to form a stronger overall model⁷. The base learners may be built from different

⁴De Ville 2013.

⁵Loh 2011.

⁶Islam n.d.

⁷Lu and Zhou 2022.

machine learning algorithms, such as decision trees or neural networks, although many ensemble methods also use multiple learners of the same type. By combining several learners, the errors of one learner may be compensated for by the others, which can improve predictive performance and increase prediction accuracy. As a result, the overall performance of an ensemble model is often better than a single individual model⁸.

Bagging and **boosting** are two common techniques in ensemble learning. Both methods combine several models to improve prediction performance, but they work in different ways.

In bagging, the models are trained independently. Each model is trained on a different bootstrap sample, which means that several random samples are drawn from the original dataset. Each model produces its own prediction, and the final prediction is obtained by averaging the individual predictions in regression tasks or by majority voting in classification tasks.

In boosting, models are trained sequentially. Each new model attempts to correct the prediction errors made by the previous models. As a result, later models place greater emphasis on observations that were previously difficult to predict.

In simple terms, bagging builds several independent models and combines their results, while boosting builds models step by step so that the ensemble gradually improves⁹.

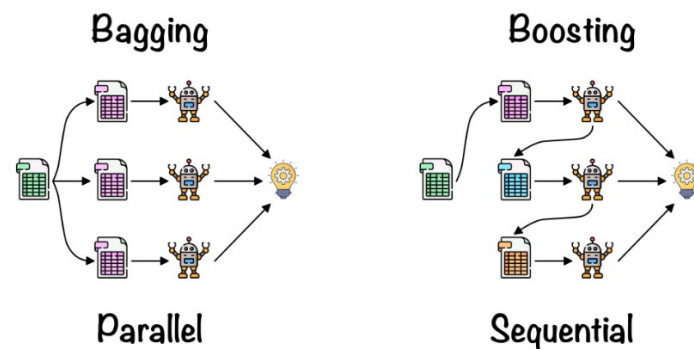


Figure 2: Bagging and Boosting.¹⁰

2.1.3 Gradient Boosting

Gradient boosting is an ensemble learning method based on the idea of boosting. It builds a strong predictive model by combining a sequence of weak learners, which are usually decision trees. These weak learners are trained sequentially, so that each new learner is added to improve the prediction errors made by the existing model. In general, boosting methods focus on reducing previous prediction errors. However, gradient boosting defines these errors through a differentiable loss function and improves the model by minimizing this loss function.

⁸Sagi and Rokach 2018.

⁹Lu and Zhou 2022.

¹⁰López 2017.

Therefore, gradient boosting can be understood as an optimization problem. Given a training dataset with n observations, the objective is to find a function F that minimizes the total loss between the true target values and the predicted values:

$$\hat{F} = \arg \min_F \sum_{i=1}^n l(y_i, F(x_i)), \quad (1)$$

where y_i is the true target value, $F(x_i)$ is the predicted value, and $l(y_i, F(x_i))$ is the loss function¹¹.

The term gradient refers to the use of gradients in the learning process. A gradient shows the direction in which the loss function increases the fastest. Therefore, to minimize the loss, the model should be updated in the opposite direction, which is called the negative gradient. This idea is related to gradient descent, an optimization method used to find the minimum of a loss function. The negative gradient at iteration m can be written as:

$$r_{im} = - \left[\frac{\partial l(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}, \quad (2)$$

where r_{im} represents the negative gradient for observation i at iteration m , and $F_{m-1}(x)$ is the current model. In gradient boosting, a new weak learner is then trained to approximate these negative gradients. The model is updated by adding this new learner to the existing model:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x), \quad (3)$$

where $F_m(x)$ is the updated model, $h_m(x)$ is the new weak learner, and η is the learning rate. The learning rate controls how large each update step is¹².

In gradient boosting, this process is applied sequentially. Each new weak learner approximates the negative gradient of the loss function and is added to the existing model. As a result, the model is improved step by step in a direction that reduces the prediction error and moves the objective function toward a lower value.

2.2 Definition and Methodology of XGBoost

Extreme Gradient Boosting, commonly known as XGBoost, was introduced by Tianqi Chen and has become a widely used machine learning algorithm because of its high predictive performance, scalability, and computational efficiency. XGBoost belongs to the family of tree-based ensemble learning methods and can be understood as an optimized implementation of gradient boosting. In this approach, multiple weak learners, usually decision trees, are trained sequentially. Each new tree is added to improve the errors made by the previous trees, so the overall model becomes progressively stronger. XGBoost is commonly applied to supervised learning tasks, including regression and classification. The algorithm is able to capture complex non-linear relationships between input features and the target variable. In addition, it includes regularization techniques that help reduce overfitting and improve generalization performance. XGBoost also uses computational optimizations,

¹¹Bentéjac, Csörgő, and Martínez-Muñoz 2021, p. 1940–1941.

¹²Natekin and Knoll 2013, p. 2–5.

such as parallel processing, which make it efficient for large datasets. As an ensemble learning method, XGBoost combines the predictive outputs of many individual decision trees into one final model prediction¹³.

The methodology of XGBoost is based on an additive ensemble learning strategy. The model starts with an initial prediction and then sequentially adds decision trees, which act as weak learners. Each new tree is added to improve the predictions made by the existing ensemble. In simple terms, the new tree focuses on the remaining prediction errors from the previous trees. Through this boosting process, XGBoost gradually improves the overall prediction performance by minimizing a specified loss function¹⁴.

For a given observation, the final prediction is obtained by summing the contributions of all trees in the ensemble. The prediction for observation i can be written as:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i).$$

Here, $\hat{y}_i$ denotes the predicted value for observation i , K is the total number of trees, x_i is the input feature vector, and f_k represents the k -th decision tree. Each tree maps an observation to a specific leaf node, and the value stored in that leaf node becomes the contribution of that tree to the final prediction¹⁵.

The training process follows an additive learning procedure. At each iteration, a new tree is added to the existing model. If the prediction after $t - 1$ trees is denoted by $\hat{y}_i^{(t-1)}$, then the prediction after adding the t -th tree becomes:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i).$$

This formula shows that the prediction is updated step by step. The new tree $f_t(x_i)$ is not trained independently, but is added to improve the current prediction of the ensemble. Therefore, XGBoost builds the final model sequentially, where each additional tree contributes to reducing the remaining prediction error and improving the model performance¹⁶.

To understand the optimization process behind XGBoost, it is useful to examine its objective function. The objective function consists of two main components: the training loss and the regularization term. It can be written as:

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k). \quad (4)$$

Here, l is a loss function that measures the difference between the predicted value $\hat{y}_i$ and the actual target value y_i . For example, in a regression task, a common loss function is the Mean Squared Error (MSE):

$$l(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2.$$

The term $\Omega(f_k)$ represents the regularization term, which penalizes the complexity of each tree. Intuitively, the objective function shows that XGBoost not only aims

¹³Quinto 2020, p. 10.

¹⁴Quinto 2020, p. 10.

¹⁵Chen and Guestrin 2016, p. 786.

¹⁶Chen and Guestrin 2016, p. 786.

to reduce prediction error but also controls the complexity of the model. This helps prevent overfitting and improves the generalization ability of the model¹⁷.

Based on the additive learning strategy described above, each iteration adds a new tree to minimize this objective function. In this way, XGBoost searches for a tree structure and corresponding leaf values that reduce the loss while keeping the model sufficiently simple through regularization.

Over n training samples, Equation 4 can then be rewritten for iteration t as:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t). \quad (5)$$

Here, f_t represents the new tree added to the model at iteration t , and $\hat{y}_i^{(t-1)}$ denotes the prediction obtained from the previous $t - 1$ trees. The purpose of adding $f_t(x_i)$ is to improve the current prediction by reducing the remaining prediction error. Therefore, XGBoost builds the model in an additive manner, where each new tree contributes an additional correction to the existing ensemble¹⁸.

Unlike ordinary parametric models, where the model parameters are usually continuous numerical values, XGBoost optimizes both the structure of decision trees and their leaf weights. Since the tree structure is discrete and difficult to optimize directly, XGBoost uses a second-order Taylor approximation of the loss function with respect to the current prediction¹⁹. This approximation transforms the objective into a form that can be optimized more efficiently using gradient and Hessian information. The second-order Taylor approximation of the objective function is written as :

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t), \quad (6)$$

where g_i and h_i are the first-order and second-order gradient statistics of the loss function, respectively. They are defined as:

$$g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}. \quad (7)$$

The first-order gradient g_i indicates the direction in which the prediction should be adjusted, while the second-order gradient h_i , also called the Hessian, describes the curvature of the loss function. By using both gradient and Hessian information, XGBoost can estimate more accurately how a new tree should improve the current model²⁰.

Since the term $l(y_i, \hat{y}_i^{(t-1)})$ is constant with respect to the new tree f_t , it can be removed from the optimization at iteration t . This gives the simplified objective function:

¹⁷Chen and Guestrin 2016, p. 786.

¹⁸Chen and Guestrin 2016, p. 786.

¹⁹Chen and Guestrin 2016, p. 786.

²⁰Chen and Guestrin 2016, p. 786.

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t). \quad (8)$$

To describe the structure of a tree, XGBoost defines a function $q(x)$, which assigns each input observation x_i to a specific leaf. Let $I_j = \{i \mid q(x_i) = j\}$ denote the set of observations assigned to leaf j . This means that all observations in the same leaf share the same leaf weight w_j . For example, if observations x_1 , x_4 , and x_5 are assigned to leaf 2, then $I_2 = \{1, 4, 5\}$.

The regularization term in XGBoost is commonly written as:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2. \quad (9)$$

In this formula, T is the number of leaves in the tree, w_j is the weight of leaf j , γ controls the penalty for adding more leaves, and λ controls the penalty on the leaf weights. The term γT penalizes overly complex trees, while the second term penalizes large leaf weights. This regularization helps reduce overfitting and improves the stability of the model²¹.

By substituting the regularization term into the simplified objective function and grouping the observations according to their assigned leaves, the objective function can be transformed as follows²²:

$$\begin{aligned} \tilde{\mathcal{L}}^{(t)} &= \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \\ &= \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T. \end{aligned} \quad (10)$$

For simplicity, the sum of gradients and Hessians in leaf j can be defined as:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i. \quad (11)$$

With these definitions, the optimal weight of leaf j is given by:

$$w_j^* = -\frac{G_j}{H_j + \lambda}. \quad (12)$$

This formula shows that the optimal leaf weight depends on the accumulated gradient G_j , the accumulated Hessian H_j , and the regularization parameter λ . The negative sign indicates that the update is made in the direction that reduces the loss²³.

After substituting the optimal leaf weights back into the objective function, the scoring function for a given tree structure q becomes:

²¹Chen and Guestrin 2016, p. 786.

²²Chen and Guestrin 2016, p. 787.

²³Chen and Guestrin 2016, p. 787.

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T. \quad (13)$$

Based on Chen and Guestrin²⁴, this scoring function is used to measure the quality of a tree structure. The goal is to find a tree structure that minimizes the regularized objective function. Therefore, XGBoost not only tries to reduce the training loss but also controls the complexity of the tree to reduce the risk of overfitting.

XGBoost also uses a split-gain formula to decide whether a node should be split. The gain from splitting a node into a left and right child is calculated as:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma. \quad (14)$$

Here, G_L and H_L are the gradient and Hessian sums for the left child, while G_R and H_R are the gradient and Hessian sums for the right child. G and H are the corresponding sums before the split. The parameter γ is subtracted as a penalty for creating an additional split. A split is accepted only when it improves the objective function sufficiently²⁵.

In addition to this mathematical formulation, XGBoost includes several practical techniques to improve performance and reduce overfitting. One important technique is shrinkage, also known as the learning rate. Instead of adding the full output of each new tree to the model, XGBoost scales the contribution of each tree by a small learning rate. This makes the learning process slower but often improves generalization. Another technique is subsampling, where only a portion of the observations or features is used when constructing each tree. This reduces variance and makes the model less dependent on specific observations or features.

Overall, XGBoost is an efficient implementation of gradient-boosted decision trees. Its methodology is based on additive tree modeling, regularized objective optimization, second-order Taylor approximation, optimal leaf-weight calculation, and gain-based tree splitting. These components allow XGBoost to build accurate predictive models while controlling model complexity and improving computational efficiency.

²⁴Chen and Guestrin 2016.

²⁵Chen and Guestrin 2016, p. 787.

3 Post Hoc Explainability

Understanding how and why a machine learning model produces specific predictions is often as important as achieving high predictive accuracy, particularly in applications where transparency, trust, and accountability are required. Higher predictive accuracy is often achieved through increased structural complexity of the model, especially in the case of black-box models such as ensemble methods or neural networks that even experts struggle to interpret. This situation creates significant tension between predictive accuracy and the comprehensibility of explanations²⁶.

To deal with this challenge, post-hoc interpretability methods have been developed to provide explanations for black-box models without requiring insight into their internal structure. These methods are model-agnostic and can therefore be applied flexibly across different types of machine learning models. Post-hoc approaches allow various representations of model behavior and provide multiple complementary perspectives on the same predictive mechanism. As a result, they can uncover relationships in data that may not be captured by traditional models. The combination of different post-hoc techniques enables meaningful and comprehensible explanations even for highly complex predictive systems²⁷.

Post-hoc interpretability methods can generally be divided into global and local approaches. Global methods aim to describe the overall behavior of a model across the entire dataset, while local methods focus on explaining individual predictions²⁸. SHAP (Shapley Additive Explanations) connects these two perspectives by assigning consistent importance values to features at the individual prediction level while also allowing these values to be aggregated in order to analyze general model behavior. A detailed discussion of SHAP will be provided later.

The correct interpretation of predictive model outputs plays a crucial role in practical applications. Traditionally, simpler models, particularly linear models, have been preferred because their structure allows direct interpretation of model parameters. However, such models often lack the flexibility required to capture complex nonlinear relationships in modern high-dimensional datasets. With the increasing availability of big data, more complex machine learning models are increasingly being applied, which further strengthens the need for reliable interpretation techniques. While complex models often cannot be summarized concisely at a global level, their behavior can be better understood through local explanations of individual predictions. Model-agnostic interpretability methods make use of this idea by approximating complex models locally, which makes it possible to gain interpretable insights without substantially reducing predictive performance.

3.1 Shapley Additive Explanations (SHAP)

Feature contributions to a model prediction can vary considerably, and in many cases features do not act independently but influence the prediction through in-

²⁶Lundberg and S.-I. Lee 2017, p. 1.

²⁷Kamath and Liu 2021, p. 167.

²⁸Kamath and Liu 2021, p. 167.

teractions with other features²⁹. Understanding these contributions is important because accurate predictions alone are often not sufficient; they also need to be interpretable. Reliable explanations help build trust in a model, provide insights into possible model improvements, and support a better understanding of the underlying prediction process³⁰.

A challenge in explaining complex predictive models is that they generally cannot handle arbitrary perturbations or incomplete feature subsets in a meaningful way when these are applied directly to the original input space. In practice, “removing” a feature is often not straightforward. For instance, a continuous variable cannot simply be set to an absent state, categorical variables may lose their valid encoding if removed, and models operating on structured data such as images or text typically require inputs in a coherent form. As a result, perturbing the original input may create unrealistic or invalid samples that the model was never trained to process, which makes direct feature attribution difficult.

To address this issue, SHAP builds on the idea of an additive feature attribution model, where a complex prediction model is approximated locally by a simpler and interpretable explanation model. The original input vector x is transformed into a simplified binary representation $x' \in \{0, 1\}^n$, in which each element indicates whether a feature is included in a coalition or not. This simplified representation is connected to the original input space through a mapping function $x = h_x(x')$.

The explanation model is then expressed as

$$g(x') = \Phi_0 + \sum_{i=1}^n \Phi_i x'_i \quad (15)$$

where $g(x')$ denotes the explanation model, Φ_0 represents a baseline value, often taken as the average model prediction, and Φ_i captures the contribution of feature i to the prediction. In this way, the original model $f(x)$ is approximated locally through $g(x')$, so that a prediction can be decomposed into additive contributions from individual features³¹. This additive framework forms the conceptual basis of SHAP and provides the link between model explainability and ideas originating from cooperative game theory.

The theoretical roots of SHAP lie in **game theory**, particularly in the concept of the Shapley value, introduced by Lloyd Shapley in 1953³². In cooperative games, a set of players forms coalitions to generate a total payoff, and the central question is how to distribute this payoff fairly among the players according to their individual contributions. Shapley addressed this by defining a value allocation method based on fairness axioms, namely *efficiency*, *symmetry*, *missingness*, and *additivity*. These axioms ensure that all contributions are fully allocated, identical contributors receive identical rewards, irrelevant players receive zero contribution, and contributions behave consistently across combined games. This fairness concept translates naturally into machine learning by viewing features as “players”, the model prediction as the

²⁹Ponce-Bobadilla et al. 2024, p. 2.

³⁰Lundberg and S.-I. Lee 2017, p. 1.

³¹Lundberg and S.-I. Lee 2017, p. 2.

³²Shapley et al. 1953.

“payout” and feature subsets as coalitions contributing to the prediction³³.

Inspired by game theory, SHAP interprets prediction explanations as a cooperative game in which each feature contributes marginally to the final model output. The contribution assigned to each feature is given by its **Shapley value**, defined as the weighted average marginal contribution of that feature across all possible coalitions of features:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f(S \cup \{i\}) - f(S)] \quad (16)$$

where N denotes the full set of features, S is a subset (coalition) excluding feature i , $|S|$ is the number of elements (features) in coalition S and $f(S)$ is the value function representing the model output based only on features in coalition S . The term $f(S \cup \{i\}) - f(S)$ measures the marginal contribution of feature i when added to coalition S , while the factorial weighting ensures that all possible feature orderings are considered fairly. SHAP adopts these Shapley values as the unique theoretically grounded feature attributions satisfying desirable properties for explainability. In the SHAP framework, the additive explanation model and Shapley values are unified, so each ϕ_i in the mapping function corresponds precisely to the Shapley value of feature i . This provides both interpretability and theoretical guarantees. A Shapley value is not merely a sensitivity measure but an expectation over all possible ways a feature can contribute to a prediction. The prediction of the additive model can therefore be expressed as the sum of all Shapley values and the expected model prediction, i.e.,³⁴

$$f(x) = \sum_{i=1}^n \phi_i(f) + E[f(x)]. \quad (17)$$

A positive SHAP value indicates that a feature pushes the prediction upward relative to the baseline, while a negative SHAP value lowers the prediction. Because these contributions sum exactly to the prediction difference from the baseline, SHAP offers both local interpretability for individual predictions and, through aggregation, global insight into feature importance and interactions. SHAP (*SHapley Additive exPlanations*) provides a standardized measure of feature relevance. A SHAP value corresponds to the Shapley value of a conditional expected value of f , taking into account the possible coalitions, i.e.,

$$f(X) = E[f(X) \mid S]. \quad (18)$$

Thus, the method assigns a value to each feature that describes the change in the expected model prediction when that feature is included. The SHAP value describes how to get from the base value $\mathbb{E}[f(0)]$, which would be assumed if no features were known, to the output of the model $f(X)$ ³⁵.

Intuitively, each feature can either be included in or excluded from a coalition. This results in 2^n possible subsets of features. When focusing on a specific feature x_i , the number of subsets that do not include x_i is 2^{n-1} , where n denotes the total number of features³⁶. This exponential growth implies that the computational complexity

³³Ponce-Bobadilla et al. 2024, p. 2.

³⁴Kamath and Liu 2021, p. 191.

³⁵Lundberg and S.-I. Lee 2017, p. 5.

³⁶Gianfagna and Di Cecco 2021, p. 99.

of calculating Shapley values increases rapidly with the number of features. Consequently, applying Formula 16 to compute SHAP values for feature contributions in a machine learning (ML) model becomes computationally infeasible for large feature sets. This issue is particularly pronounced in time-series data, where the number of dependent features can grow substantially when temporal information is represented at fine-grained resolutions, such as hourly or second-level intervals. Depending on the type of model (ANN, decision trees, etc.), there are various methods for approximating SHAP values. KernelSHAP and TreeSHAP are explained in more detail below³⁷.

3.2 KernelSHAP

KernelSHAP is an approximation method for computing Shapley values that can be applied to any supervised machine learning model³⁸. To understand the underlying idea of KernelSHAP, it is useful to first introduce the concept of local surrogate models and the method LIME, upon which KernelSHAP is conceptually based³⁹.

In model interpretability, a distinction is made between global and local explanations. Global explanations aim to describe the overall behavior of a model, whereas local explanations focus on individual predictions and seek to explain why a model produced a specific output for a given data instance⁴⁰.

Local surrogate models approximate the behavior of a complex black-box model f in the vicinity of a specific data instance x by means of a simpler and interpretable model g . This approach is based on the principle of *local fidelity*, which requires that the surrogate model accurately reflects the behavior of the original model in the neighborhood of x , without necessarily capturing its global structure⁴¹.

LIME (*Local Interpretable Model-Agnostic Explanations*) is a prominent example of such a surrogate-based approach. It constructs an interpretable model (e.g., a linear model or decision tree) that approximates the predictions of the black-box model locally⁴². To obtain this surrogate model, LIME solves the following optimization problem:

$$\mathcal{L}(g) = L(f, g, \pi_x) + \Omega(g), \quad (19)$$

where the objective is to identify an interpretable model g that minimizes the locally weighted loss $L(f, g, \pi_x)$ while controlling model complexity through the regularization term $\Omega(g)$. The proximity function $\pi_x(z)$ assigns higher weights to perturbed samples z that lie closer to the instance x and lower weights to those that are farther away. These perturbed samples are generated from the original instance x , for example through random modifications of its feature values such as Gaussian noise, and are used to evaluate the black-box model locally around x ⁴³. In this way, the surrogate model g approximates the behavior of the original model f in the neighborhood of the instance being explained. The loss term measures how accurately

³⁷Ponce-Bobadilla et al. 2024, p. 4.

³⁸Ponce-Bobadilla et al. 2024, p. 4.

³⁹Gianfagna and Di Cecco 2021, p. 101.

⁴⁰Kamath and Liu 2021, p. 12.

⁴¹Kamath and Liu 2021, p. 198.

⁴²Gianfagna and Di Cecco 2021, p. 99.

⁴³Kamath and Liu 2021, p. 201.

g reproduces the local behavior of f , whereas the complexity term $\Omega(g)$ penalizes overly complex explanatory models in order to preserve interpretability. Thus, the more complex the surrogate model g , the larger the value of Ω . By constraining this complexity term, LIME seeks to improve interpretability without substantially compromising local fidelity⁴⁴.

Similar to many local explanation approaches, LIME relies on a mapping function that transforms an instance into an interpretable representation, often using a binary vector to indicate the presence or absence of features. However, this simplification may lead to the loss of important information from the original data representation and can therefore reduce the fidelity of the explanation⁴⁵.

LIME is characterized by its flexibility and its ability to produce explanations that are easy to understand. Nevertheless, it also has notable limitations, particularly when explaining models with strongly nonlinear behavior in the local neighborhood of the instance of interest. Because the perturbed samples z are synthetic variations rather than real observed instances, they do not always fully capture the characteristics of the original instance x . As a result, LIME may generate different explanations for instances that are very similar to one another⁴⁶.

As already explained, the computation of Shapley values is challenging because it requires the evaluation of the marginal contribution of each feature across all possible coalitions of the remaining features. Since the number of such coalitions grows exponentially with the number of features, an exact computation quickly becomes infeasible. Efficient approximation methods are therefore essential⁴⁷. KernelSHAP provides such an approximation and is the only local linear surrogate method specifically developed for estimating Shapley values⁴⁸. Similar to LIME, KernelSHAP uses a weighting function π_x to emphasize information in the local neighborhood of the instance to be explained. It approximates Shapley values by sampling feature coalitions S and fitting a linear additive surrogate model to the outputs of the original model f ⁴⁹.

In KernelSHAP, the estimation of Shapley values can be formulated as the fitting of a weighted linear regression model whose coefficients correspond to the Shapley values. This is achieved by minimizing the following objective function:

$$\Psi = \min_{\phi_1, \dots, \phi_M} \left[\sum_K \left(f(S) - \sum_{x_j \in S} \phi_j \right)^2 \pi(S) \right], \quad (20)$$

where the weighting function is defined as

$$\pi(S) = \frac{n-1}{\binom{n}{|S|} |S| (n-|S|)}. \quad (21)$$

Unlike in LIME, the weighting function π in KernelSHAP depends on feature coalitions

⁴⁴Gianfagna and Di Cecco 2021, p. 101.

⁴⁵Kamath and Liu 2021, p. 201.

⁴⁶Kamath and Liu 2021, p. 202.

⁴⁷Covert and S. Lee n.d., p. 3.

⁴⁸Gianfagna and Di Cecco 2021, p. 101.

⁴⁹Kamath and Liu 2021, p. 193.

tions S rather than on simplified data instances. Equation 21 indicates that particularly small and particularly large coalitions receive higher weights because they are especially informative for assessing feature relevance. Thus, a coalition consisting of a single feature can be as informative as a coalition containing all but one feature⁵⁰. Because KernelSHAP estimates the Shapley values by fitting a linear surrogate model under a quadratic loss function, the optimization problem can be solved using weighted linear regression⁵¹.

3.3 TreeSHAP

TreeSHAP is an efficient algorithm for computing SHAP values for tree-based models, such as decision trees, random forests, and gradient-boosted trees, including XGBoost. In the general SHAP framework, the exact computation of Shapley values requires evaluating the marginal contribution of each feature over all possible feature coalitions. Therefore, when the number of input features is large, direct computation becomes computationally expensive⁵². TreeSHAP addresses this problem by exploiting the internal structure of decision trees. Instead of explicitly evaluating all possible feature subsets, TreeSHAP uses the paths of the tree to compute the contribution of each feature efficiently. Each path from the root node to a leaf represents a sequence of splitting conditions that determines whether a data point reaches a particular leaf. By tracking how feature coalitions affect the probability of following these paths, TreeSHAP can compute exact SHAP values for tree models in polynomial time⁵³.

Consider a tree model with K leaves. Let v_k denote the prediction value associated with leaf k , and let P_k denote the set of internal nodes along the path P_k from the root to leaf k . At each node $j \in P_k$, the tree splits on feature d_j . Let $T_{k,j}$ denote the condition that must be satisfied at node j in order to continue along the path to leaf k . For a coalition of observed features S , the path weight $W_{k,S}$ represents the probability that an observation x reaches leaf k when only the features in S are known. It can be written as:

$$W_{k,S} = \prod_{j \in P_k} \begin{cases} 1, & \text{if } d_j \in S \text{ and } x_{d_j} \in T_{k,j}, \\ 0, & \text{if } d_j \in S \text{ and } x_{d_j} \notin T_{k,j}, \\ R_{k,j}, & \text{if } d_j \notin S, \end{cases} \quad (22)$$

where $R_{k,j}$ denotes the proportion of training observations that follow the branch from node j toward leaf k . More precisely, $R_{k,j}$ is given by the ratio between the number of training observations reaching the next node on the path P_k and the number of training observations reaching node j . Thus, if the splitting feature d_j is observed and its value satisfies the path condition, the observation follows the path with weight one. If the condition is not satisfied, the path receives weight zero. If the feature is not included in the coalition S , TreeSHAP uses the empirical training proportion $R_{k,j}$ to represent the uncertainty about which branch should be followed⁵⁴.

⁵⁰Kamath and Liu 2021, p. 193.

⁵¹Lundberg and S.-I. Lee 2017, p. 6.

⁵²Lundberg, Erion, et al. 2020, p. 5.

⁵³Gianfagna and Di Cecco 2021, p. 107, 108.

⁵⁴Yang 2021, p. 3, 4.

Using these path weights, the expected prediction of the tree conditional on the feature coalition S can be expressed as

$$f_S(x) = \sum_{k=1}^K W_{k,S} v_k. \quad (23)$$

Substituting this expression into the general Shapley value formula gives the SHAP value of feature i for a single tree:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} \sum_{k=1}^K (W_{k, S \cup \{i\}} - W_{k,S}) v_k. \quad (24)$$

This expression shows that the contribution of feature i is obtained by comparing the tree prediction when feature i is included in the coalition with the tree prediction when it is excluded. TreeSHAP computes these quantities efficiently by using the recursive structure of tree paths instead of explicitly evaluating all possible subsets of features⁵⁵.

For tree ensembles, such as XGBoost, LightGBM, and random forests, the final prediction is usually obtained by adding the predictions of many individual trees. Therefore, TreeSHAP computes the SHAP values for each tree and then sums these values across all trees in the ensemble. If the model consists of T trees, the SHAP value of feature j for the full ensemble can be written as

$$\phi_j^{\text{ensemble}} = \sum_{t=1}^T \phi_j^{(t)}, \quad (25)$$

where $\phi_j^{(t)}$ denotes the SHAP value of feature j in tree t . This additive property makes TreeSHAP particularly suitable for tree ensemble models, because the explanation of the full prediction can be decomposed into the explanations generated by the individual trees.

The main advantage of TreeSHAP is computational efficiency. While the general SHAP formulation requires evaluating a large number of possible feature coalitions, TreeSHAP applies a dynamic programming procedure over tree paths to compute exact SHAP values efficiently. As a result, TreeSHAP preserves the theoretical interpretation of SHAP values while adapting their computation to the specific structure of decision trees⁵⁶.

⁵⁵Yang 2021, p. 4.

⁵⁶Lundberg, Erion, et al. 2020.

4 Causal Inference Framework

Predictive modeling and causal inference represent two related but conceptually distinct perspectives in statistical learning. The conflation of prediction and causation still occurs in many studies⁵⁷. Although both approaches are widely used in empirical research, they serve different purposes and address different scientific questions. Predictive modeling aims to estimate outcome values for new or future observations based on observed input variables. In contrast, causal inference seeks to determine whether changing a treatment, exposure, or explanatory variable would lead to a change in the outcome⁵⁸. Consequently, prediction focuses on forecasting what will happen, whereas causal inference focuses on understanding what would happen under a hypothetical intervention⁵⁹.

This distinction is important because machine-learning models are typically designed to learn patterns and statistical associations in observed data in order to maximize predictive performance. As a result, a model may rely heavily on a variable because it improves prediction accuracy, even if that variable is not a causal determinant of the outcome. Consequently, explaining a predictive model is fundamentally different from estimating a causal effect. A predictive explanation describes how a model generates its output, whereas a causal explanation requires assumptions regarding interventions, confounding factors, and counterfactual outcomes⁶⁰.

This distinction becomes particularly relevant when interpreting SHAP values. As discussed in the previous section, SHAP is widely used to explain complex machine-learning models by assigning an attribution value to each feature for a given prediction⁶¹. In this way, SHAP identifies the features that contribute most strongly to model predictions. However, SHAP explains the behavior of the predictive model rather than the causal mechanisms underlying the data-generating process. A high SHAP value indicates that a feature has a strong influence on the model prediction for a particular observation or across a dataset. It does not necessarily imply that changing the feature would cause a corresponding change in the real-world outcome⁶².

This limitation reflects the well-known principle that statistical association alone is insufficient to establish causality⁶³. A variable may be associated with an outcome because of *confounding*, *reverse causation*, or *indirect relationships*. For example, during periods of hot weather, people tend to consume more ice cream and engage in swimming activities more frequently, while drowning incidents may also increase. However, this does not imply that ice-cream consumption causes drowning. Instead, hot weather acts as a common cause that influences both behaviors. Similarly, a feature may appear important for prediction because it is statistically associated with the outcome, yet this association alone does not demonstrate a causal effect.

Therefore, a machine-learning model can achieve strong predictive performance

⁵⁷Ramspek et al. 2021, p. 1.

⁵⁸Shmueli 2010, p. 2.

⁵⁹Pearl, Glymour, and Jewell 2016.

⁶⁰Shmueli 2010, p. 8.

⁶¹Lundberg and S.-I. Lee 2017.

⁶²Ng et al. 2025.

⁶³Altman and Krzywinski 2015.

without providing valid causal conclusions. Likewise, an interpretable predictive model may reveal which variables are important for prediction, but such importance should not be confused with causal relevance. SHAP answers the question of which features the model uses when generating a prediction. It does not answer the causal question of what would happen if a variable were externally manipulated while all relevant confounding factors were adequately controlled⁶⁴.

Consequently, the discussion naturally progresses from explainable artificial intelligence to causal inference. While SHAP helps improve the transparency and interpretability of black-box prediction models, causal inference provides a framework for estimating cause-and-effect relationships. Rather than explaining how a model arrives at a prediction, causal inference seeks to determine whether a treatment or variable has a genuine effect on an outcome. Answering such questions requires additional assumptions and methodological tools, including the identification of confounders, the specification of treatment and outcome variables, and the consideration of counterfactual outcomes. Therefore, while SHAP can identify variables that are important for prediction, causal inference is required to determine whether these variables represent genuine causal drivers of the outcome.

4.1 Directed Acyclic Graphs (DAG)

Causal graphs provide a useful framework for representing assumptions about causal relationships between variables. A widely used graphical representation in causal inference is the directed acyclic graph (DAG). In a DAG, variables are represented as nodes, while directed arrows represent assumed causal relationships between variables. A DAG can therefore be understood as a graphical representation of how variables are assumed to influence one another. In the context of treatment effect estimation, the main interest is usually the causal effect of a treatment or intervention variable D on an outcome variable Y , while other variables X may act as covariates, confounders, mediators, or colliders⁶⁵.

It is important to note that a DAG does not prove causal relationships by itself. Instead, it represents assumptions about the causal structure of the data-generating process. These assumptions are necessary because causal effects cannot be identified from observed statistical associations alone. In observational data, researchers observe relationships between variables, but they usually do not observe what would have happened under a different intervention. Therefore, DAGs are useful because they make causal assumptions explicit and help determine which variables should be adjusted for when estimating a causal effect.

The term directed means that the arrows have a specific direction, indicating the assumed direction of causal influence. For example, the arrow $D \rightarrow Y$ represents the assumption that the treatment D has a causal effect on the outcome Y . A causal effect can be represented by a direct path, such as $D \rightarrow Y$, or by an indirect path through another variable, such as $D \rightarrow X \rightarrow Y$. The term acyclic means that the graph does not contain feedback loops. In other words, a variable cannot cause itself

⁶⁴Shmueli 2010.

⁶⁵Cunningham 2021, p. 97–118.

directly or indirectly through a closed causal path⁶⁶.

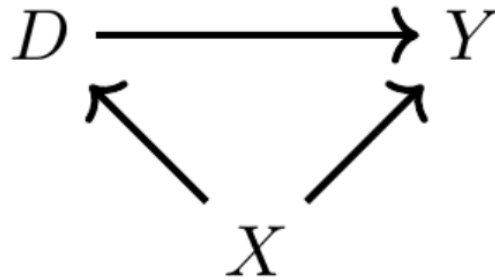


Figure 3: Confounding structure in a DAG.

Figure 3 shows a simple DAG with three variables: treatment D , outcome Y , and covariate X . The directed path $D \rightarrow Y$ represents the causal effect of interest. However, the graph also contains the path $D \leftarrow X \rightarrow Y$. This path is called a backdoor path because it connects D and Y through a common cause X , rather than through the causal effect of D on Y . In this case, X is a confounder because it affects both the treatment and the outcome⁶⁷.

Confounding is important because it can create a statistical association between treatment and outcome that does not fully represent the causal effect of the treatment. In Figure 3, differences in Y may be caused by changes in D , but they may also be partly explained by differences in X . Therefore, if X is not controlled for, the estimated relationship between D and Y may be biased. To estimate the causal effect of D on Y , the backdoor path $D \leftarrow X \rightarrow Y$ *must be blocked*, usually by conditioning on appropriate confounders⁶⁸.

Returning to the previous ice-cream example, the relationship between ice-cream consumption and drowning incidents illustrates confounding. During hot weather, people may consume more ice cream and also swim more frequently, which can increase the number of drowning incidents. As a result, ice-cream consumption and drowning incidents may be statistically associated. However, this does not mean that ice-cream consumption causes drowning. Instead, hot weather acts as a confounder because it influences both ice-cream consumption and the risk of drowning. Similarly, in observational data, a variable may appear to be related to the outcome because both the treatment and the outcome are influenced by a common cause.

However, DAGs also show that not every associated variable should be controlled for. A particularly important example is collider bias. A collider is a variable that is influenced by two other variables. If two variables both affect a third variable, conditioning on this third variable can create a statistical association between the two original variables, even if no causal relationship exists between them⁶⁹.

Figure 4 shows a collider structure, where both D and Y influence X . In this case, X is a collider on the path $D \rightarrow X \leftarrow Y$. Without conditioning on X , this path is blocked. However, if the analysis controls for X , a statistical association between D

⁶⁶Cunningham 2021, p. 97–118.

⁶⁷Cunningham 2021, p. 97–118.

⁶⁸Cunningham 2021, p. 97–118.

⁶⁹Cunningham 2021, p. 97–118.

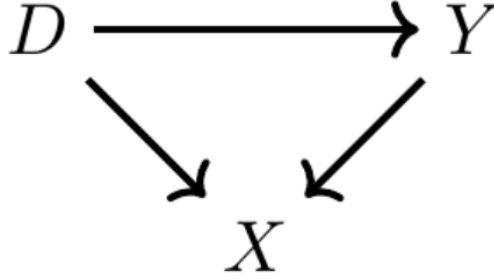


Figure 4: Collider structure in a DAG.

and Y can be introduced. Therefore, controlling for a collider can create bias rather than remove it⁷⁰.

Overall, DAGs provide a conceptual tool for connecting causal assumptions with statistical estimation. They help identify causal paths, backdoor paths, confounders, and colliders. This is important because causal estimation methods require careful decisions about which variables should be adjusted for and which variables should not be conditioned on. Methods such as propensity score adjustment and Double Machine Learning still rely on the assumption that relevant confounders are observed and appropriately accounted for. Therefore, DAGs are useful for clarifying the causal assumptions that support later statistical estimation.

4.2 Average Treatment Effect (ATE)

After the causal structure has been clarified, the next step is to define the causal quantity of interest. Such a target quantity is called a causal estimand. In many empirical applications, the main estimand is the average treatment effect (ATE). The ATE measures the average causal effect of a treatment or intervention on an outcome in a population. It answers the question of how the outcome would change on average if all units in the population received the treatment compared with if all units did not receive the treatment⁷¹.

The ATE is commonly introduced using the potential outcome framework. This framework describes causal effects by comparing two potential outcomes for the same unit: one outcome under treatment and one outcome without treatment. For each observation i , there are two potential outcomes. $Y_i(1)$ denotes the outcome that would be observed if unit i received the treatment, while $Y_i(0)$ denotes the outcome that would be observed if unit i did not receive the treatment⁷². The individual treatment effect can therefore be written as:

$$\delta_i = Y_i(1) - Y_i(0). \quad (26)$$

However, for each unit, only one of these two potential outcomes can be observed in practice. If a unit receives the treatment, $Y_i(1)$ is observed and $Y_i(0)$ remains counterfactual. If a unit does not receive the treatment, $Y_i(0)$ is observed and $Y_i(1)$

⁷⁰Cunningham 2021, p. 97–118.

⁷¹Cunningham 2021, p. 119–123.

⁷²Cunningham 2021, p. 119–123.

remains counterfactual. This is known as the fundamental problem of causal inference. Because one potential outcome is always unobserved for each unit, individual treatment effects cannot be directly observed. Therefore, causal effects such as the ATE must be estimated under additional assumptions⁷³.

Since individual treatment effects cannot generally be observed directly, causal inference often focuses on average effects. The average treatment effect is defined as:

$$ATE = \mathbb{E}[Y(1) - Y(0)] = \mathbb{E}[Y(1)] - \mathbb{E}[Y(0)]. \quad (27)$$

This estimand represents the expected difference between the potential outcome under treatment and the potential outcome without treatment. In observational studies, however, estimating the ATE requires *additional assumptions*. This is because treated and untreated units may differ from each other in important characteristics before the treatment is applied. Therefore, a simple comparison between the two groups may be biased. One common assumption is conditional exchangeability, also called unconfoundedness or selection on observables. This assumption means that, after controlling for a suitable set of covariates X , the treatment assignment can be considered independent of the potential outcomes. In simpler terms, treated and untreated units should be compared only when they have similar relevant characteristics. For example, when estimating the causal effect of aspirin, treated and untreated individuals should be comparable in important factors such as age, health condition, and other relevant medical characteristics. Under this assumption, differences in outcomes between treated and untreated units with the same covariates can be interpreted as causal effects⁷⁴.

Therefore, defining the ATE is not only a mathematical step but also a conceptual step. It clarifies which causal effect the analysis aims to estimate. This is important because different estimands answer different research questions. For example, the ATE focuses on the average effect in the entire population. In contrast, the average treatment effect on the treated (ATTE) focuses only on units that actually received the treatment, while the average treatment effect on the untreated (ATTU) focuses only on units that did not receive the treatment⁷⁵. In this thesis, the ATE is relevant because it provides a general framework for estimating the average causal effect of a treatment variable on an outcome variable.

4.3 Propensity Score

In observational data, treatment assignment is usually not random. Treated and untreated observations may differ in many characteristics, and these differences can lead to confounding. One classical approach for reducing confounding is based on the propensity score. Rosenbaum and Rubin introduced the propensity score and showed that, under suitable assumptions, conditioning on the propensity score can balance the distribution of observed covariates between treated and untreated groups⁷⁶. The propensity score is defined as the probability of receiving the treatment conditional on observed covariates. For a binary treatment variable D_i , where $D_i = 1$ means

⁷³Naimi and Whitcomb 2023.

⁷⁴Naimi and Whitcomb 2023.

⁷⁵Cunningham 2021, p. 119–123.

⁷⁶Rosenbaum and Rubin 1983, p. 41–43.

that unit i receives the treatment and $D_i = 0$ means that unit i does not receive the treatment, the propensity score can be written as:

$$e(X_i) = P(D_i = 1 \mid X_i), \quad (28)$$

where X_i denotes the observed covariates of unit i . The central idea is that instead of adjusting for many covariates directly, one can adjust for the probability of treatment assignment given these covariates. In practice, the propensity score is often estimated using logistic regression. Since the treatment variable is binary, logistic regression models the log-odds of receiving the treatment as a function of the observed covariates:

$$\log \left(\frac{e(X_i)}{1 - e(X_i)} \right) = \beta_0 + X_i \beta, \quad (29)$$

where $\frac{e(X_i)}{1 - e(X_i)}$ represents the odds of receiving the treatment, β_0 is the intercept, and β represents the coefficients of the observed covariates X_i . After estimating this logistic regression model, the predicted probability of receiving the treatment is used as the propensity score. In other words, the propensity score describes how likely a unit is to receive the treatment based on its observed characteristics⁷⁷.

Propensity scores can be used in several ways. One common approach is matching, where treated observations are compared with untreated observations that have similar propensity scores. Another approach is weighting, where observations are weighted by functions of the propensity score in order to create a pseudo-population in which treatment assignment is more balanced across covariates. Propensity scores can also be included as control variables in regression adjustment. These methods share the same general goal: to make treated and untreated groups more comparable with respect to observed confounders⁷⁸.

Despite its usefulness, the propensity score does not automatically solve the problem of causal identification. It can only adjust for observed covariates. If important confounders are unobserved, the estimated treatment effect may still be biased. Therefore, propensity-score-based methods rely on the assumption that all relevant confounders have been measured and correctly included in the adjustment set⁷⁹. This connects back to the role of DAGs, because causal graphs help determine which variables are potential confounders and should be considered in the adjustment strategy.

Propensity score methods are important in the development of causal inference because they provide a structured way to estimate causal effects from observational data. However, in modern data science applications, the number of covariates can be large and the relationships between treatment, covariates, and outcome can be complex. This creates challenges for traditional estimation methods and motivates the use of machine-learning-based approaches. These challenges are especially relevant for Double Machine Learning, which extends the idea of estimating nuisance functions, such as the propensity score and outcome model, using flexible machine-learning methods while still aiming to obtain valid causal effect estimates.

⁷⁷Rosenbaum and Rubin 1983, p. 42-43.

⁷⁸Haukoos and Lewis 2015.

⁷⁹Haukoos and Lewis 2015.

5 Double Machine Learning

5.1 Challenges in High-Dimensional Causal Inference

In modern empirical applications, causal-effect estimation often involves a large number of observed covariates and potentially complex relationships among the treatment, the outcome, and the control variables. Omitting relevant confounders may lead to biased treatment-effect estimates, while including many irrelevant variables can increase estimation uncertainty and reduce model stability. The central challenge is therefore to adjust for the variables required for causal identification without imposing an excessively restrictive functional form⁸⁰.

This challenge is particularly important because prediction and causal inference have different objectives. A predictive model is primarily designed to minimize prediction error, whereas causal inference aims to estimate a specific causal or structural parameter. A variable that contributes little to predicting the outcome may nevertheless be an important confounder if it is strongly related to the treatment. Consequently, a variable-selection procedure designed only for prediction may exclude variables that are necessary for valid causal-effect estimation⁸¹.

Machine-learning methods provide flexible tools for estimating complex and potentially nonlinear relationships. However, many such methods rely on regularization, variable selection, or other forms of model adaptation. Although these procedures may improve predictive performance, they can introduce estimation errors that affect the target parameter. Double Machine Learning addresses this problem by using machine learning to estimate nuisance functions while reducing the first-order impact of nuisance estimation on the target-parameter estimator⁸².

5.2 Target Parameters and Nuisance Functions

Double Machine Learning distinguishes between a low-dimensional target parameter and additional nuisance components required for its identification and estimation. The target parameter is denoted by θ_0 . Depending on the empirical model and its identifying assumptions, θ_0 may represent a treatment coefficient, an average treatment effect, or another causal or structural quantity⁸³.

The nuisance parameter is denoted by η_0 and may contain one or several unknown functions. Although these functions are not the primary objects of interest, they enter the moment condition used to identify θ_0 . In a general semiparametric framework, the target parameter satisfies

$$\mathbb{E}[\psi(W; \theta_0, \eta_0)] = 0. \quad (30)$$

Here, W denotes the observed data and $\psi(W; \theta, \eta)$ denotes a score or moment function. In high-dimensional settings, the nuisance parameter may consist of complex conditional expectation functions that cannot be represented adequately by a simple

⁸⁰Belloni, Chernozhukov, and Hansen 2014.

⁸¹Belloni, Chernozhukov, and Hansen 2014.

⁸²Chernozhukov et al. 2018.

⁸³Chernozhukov et al. 2018.

low-dimensional parametric model⁸⁴.

A central example in the DML framework is the partially linear regression model⁸⁵

$$Y = D\theta_0 + g_0(X) + U, \quad (31)$$

together with the conditional-exogeneity assumption

$$\mathbb{E}[U \mid D, X] = 0. \quad (32)$$

Here, Y denotes the outcome variable, D denotes the treatment variable, and X denotes a vector of observed control variables. The parameter θ_0 represents the constant partial effect of the treatment on the outcome under the assumptions of the partially linear model. The function $g_0(X)$ represents the structural contribution of the control variables to the outcome⁸⁶.

The treatment equation is written as

$$D = m_0(X) + V, \quad (33)$$

where

$$\mathbb{E}[V \mid X] = 0, \quad (34)$$

and

$$m_0(X) = \mathbb{E}[D \mid X]. \quad (35)$$

The residual $V = D - m_0(X)$ represents the treatment variation remaining after the component predictable from X has been removed. If D is a binary treatment variable, $m_0(X)$ is also the propensity score because

$$m_0(X) = \mathbb{P}(D = 1 \mid X). \quad (36)$$

For the partialling-out implementation of DML, the conditional outcome function is defined as

$$\ell_0(X) = \mathbb{E}[Y \mid X]. \quad (37)$$

Taking the conditional expectation of Equation 31 given X and using Equation 32 gives

$$\ell_0(X) = \theta_0 m_0(X) + g_0(X). \quad (38)$$

Therefore, $\ell_0(X)$ and $g_0(X)$ are not the same function. The structural function $g_0(X)$ appears in the partially linear outcome equation, whereas $\ell_0(X)$ is the complete conditional expectation of the outcome given X . The nuisance parameter used in the partialling-out score is consequently

$$\eta_0 = (\ell_0, m_0). \quad (39)$$

If the nuisance functions were known, the outcome and treatment could be residualized as

⁸⁴Ahrens et al. 2025, p. 2-3.

⁸⁵Chernozhukov et al. 2018, p. 2.

⁸⁶Chernozhukov et al. 2018.

$$\tilde{Y} = Y - \ell_0(X), \quad \tilde{D} = D - m_0(X). \quad (40)$$

Using the partially linear model and Equation 38, the residualized relationship becomes

$$\tilde{Y} = \theta_0 \tilde{D} + U. \quad (41)$$

The target parameter can therefore be identified from the relationship between the residualized outcome and the residualized treatment. The corresponding orthogonal score is

$$\psi_{\text{PLR}}(W; \theta, \eta) = [Y - \ell(X) - \theta(D - m(X))] [D - m(X)], \quad \eta = (\ell, m). \quad (42)$$

At the true parameter values, this score satisfies

$$\mathbb{E}[\psi_{\text{PLR}}(W; \theta_0, \eta_0)] = 0. \quad (43)$$

In practice, $\ell_0(X)$ and $m_0(X)$ are unknown and must be estimated. Flexible machine-learning methods can be used for this purpose when the relationships among X , Y , and D are nonlinear or high-dimensional. However, the causal interpretation of θ_0 continues to depend on the identifying assumptions of the partially linear model. DML addresses estimation and inference after those assumptions have been imposed; it does not establish causal identification by itself.

5.3 Regularization Bias and Overfitting Bias

A direct approach would estimate the nuisance parameter η_0 using machine learning and substitute the resulting estimate $\hat{\eta}$ into the sample moment condition. The resulting plug-in estimator satisfies

$$\frac{1}{n} \sum_{i=1}^n \psi(W_i; \hat{\theta}, \hat{\eta}) = 0. \quad (44)$$

This procedure may perform poorly because the nuisance estimate $\hat{\eta}$ is generally not equal to the true nuisance parameter η_0 . Errors may arise from regularization, variable selection, approximation error, model tuning, or overfitting. If the score is sensitive to these errors, they can propagate into the estimator of θ_0 ⁸⁷.

The effect of nuisance estimation can be illustrated using a Taylor expansion. For a simple function $f(x)$ expanded around x_0 , the expansion is

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2 + \dots \quad (45)$$

The term $f'(x_0)(x - x_0)$ is the first-order term and describes the direct linear effect of a small deviation from x_0 . If this first-order term is equal to zero, small deviations affect the function only through higher-order terms.

⁸⁷Ahrens et al. 2025, p. 2-5.

For intuition, consider a finite-dimensional representation of the nuisance parameter. A first-order expansion of the sample moment condition around the true values (θ_0, η_0) gives⁸⁸

$$\begin{aligned} \frac{1}{n} \sum_{i=1}^n \psi(W_i; \hat{\theta}, \hat{\eta}) &= \frac{1}{n} \sum_{i=1}^n \psi(W_i; \theta_0, \eta_0) \\ &+ \frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial \theta} \psi(W_i; \theta_0, \eta_0) (\hat{\theta} - \theta_0) \\ &+ \frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial \eta} \psi(W_i; \theta_0, \eta_0) (\hat{\eta} - \eta_0) + \text{higher-order terms.} \end{aligned} \quad (46)$$

Because $\hat{\theta}$ solves the sample moment condition in Equation 44, the left-hand side of Equation 46 is equal to zero. The term

$$\frac{1}{n} \sum_{i=1}^n \frac{\partial}{\partial \eta} \psi(W_i; \theta_0, \eta_0) (\hat{\eta} - \eta_0) \quad (47)$$

captures the first-order impact of nuisance estimation. If this term is not sufficiently small, the sampling distribution of $\hat{\theta}$ may differ from the ideal case in which η_0 is known, making conventional inference unreliable⁸⁹.

Regularization bias arises because flexible nuisance estimators frequently introduce systematic estimation error to control model complexity and variance. The error $\hat{\eta} - \eta_0$ is therefore not generally centred around zero in finite samples. If a non-orthogonal score is used, this estimation error may directly affect the target-parameter estimator through Equation 47⁹⁰.

Overfitting bias arises when the same observations are used both to estimate $\hat{\eta}$ and to evaluate the sample moment condition. Since $\hat{\eta}$ is then a function of the same observations W_i appearing in the score, the nuisance estimation error may be statistically dependent on the score contributions. This dependence can prevent the first-order nuisance term from becoming asymptotically negligible⁹¹.

DML addresses these two problems through two complementary components. Neyman orthogonality reduces the first-order sensitivity of the population moment to nuisance estimation errors, while cross-fitting reduces the dependence between nuisance estimation errors and the observations used to evaluate the score⁹².

5.4 Motivation for Double Machine Learning

Double Machine Learning, also known as Double/Debiased Machine Learning, provides a framework for estimating low-dimensional target parameters in the presence of high-dimensional or complex nuisance parameters. Machine-learning methods

⁸⁸Ahrens et al. 2025, p. 13-15.

⁸⁹Ahrens et al. 2025, p. 13-15.

⁹⁰Ahrens et al. 2025, p. 3-4.

⁹¹Ahrens et al. 2025, p. 3-4.

⁹²Ahrens et al. 2025.

are used to estimate the nuisance functions, but the target parameter is not obtained directly from the machine-learning predictions. Instead, it is estimated using a Neyman-orthogonal score and cross-fitted nuisance predictions⁹³.

This construction preserves the flexibility of machine learning while reducing the impact of nuisance estimation on the target-parameter estimator. Neyman orthogonality primarily addresses regularization bias, whereas cross-fitting primarily addresses overfitting bias.

5.5 Neyman Orthogonality

Neyman orthogonality is a local robustness property of the score function. It requires that small perturbations of the nuisance parameter away from its true value do not create a first-order change in the population moment condition⁹⁴.

Let η be a candidate nuisance parameter belonging to the admissible nuisance-parameter space $\mathcal{T}$. A local movement away from η_0 in the direction $\eta - \eta_0$ can be represented by

$$\eta_0 + \lambda(\eta - \eta_0), \quad (48)$$

where λ measures the size of the deviation. In addition to satisfying the population moment condition in Equation 30, a Neyman-orthogonal score satisfies

$$\left. \frac{\partial}{\partial \lambda} \mathbb{E}[\psi(W; \theta_0, \eta_0 + \lambda(\eta - \eta_0))] \right|_{\lambda=0} = 0, \quad \forall \eta \in \mathcal{T}. \quad (49)$$

Equation 49 means that the expected score is locally flat in the nuisance-parameter direction at η_0 . Consequently, small nuisance estimation errors do not affect the population moment at first order. Their remaining influence enters through higher-order terms.

Neyman orthogonality does not mean that the nuisance parameter can be estimated arbitrarily poorly. The nuisance estimator must still converge towards the true nuisance parameter sufficiently quickly. A commonly used sufficient high-level condition is

$$\sqrt{n} \|\hat{\eta} - \eta_0\|^2 \xrightarrow{p} 0, \quad (50)$$

which can equivalently be expressed as

$$\|\hat{\eta} - \eta_0\| = o_p(n^{-1/4}). \quad (51)$$

The rate $n^{-1/4}$ is a commonly cited sufficient benchmark rather than a rate automatically achieved by machine-learning methods. Whether it can be attained depends on the learner, its tuning parameters, the complexity of the nuisance functions, the dimensionality of the covariates, and structural assumptions such as sparsity or smoothness⁹⁵.

⁹³Chernozhukov et al. 2018.

⁹⁴Ahrens et al. 2025, p. 16-19.

⁹⁵Ahrens et al. 2025, p. 16-19.

Neyman orthogonality concerns sensitivity to the nuisance parameter, not sensitivity to the target parameter. The expected score must remain sufficiently responsive to changes in θ for θ_0 to be identified. Thus, the population moment is locally flat in nuisance directions but not in the target-parameter direction.

5.6 Cross-Fitting

Neyman orthogonality does not by itself eliminate the statistical dependence created when the same observations are used for both nuisance estimation and score evaluation. Cross-fitting addresses this problem by ensuring that the nuisance predictions used for a given observation are obtained from a model trained without that observation⁹⁶.

Consider an independent and identically distributed sample

$$\{W_i\}_{i=1}^n. \quad (52)$$

Let the observation indices be randomly partitioned into K non-overlapping folds of approximately equal size:

$$\{I_k\}_{k=1}^K. \quad (53)$$

For every fold I_k , the corresponding training sample is

$$I_k^c = \{1, \dots, n\} \setminus I_k. \quad (54)$$

The nuisance parameter is estimated using only the observations in I_k^c . The resulting fold-specific estimate is

$$\hat{\eta}_{-k} = \hat{\eta}(\{W_i\}_{i \in I_k^c}). \quad (55)$$

The subscript $-k$ indicates that the observations in fold I_k were excluded from nuisance estimation. The estimate $\hat{\eta}_{-k}$ is then used to evaluate the score contributions for observations in I_k .

The cross-fitted estimator is obtained by solving the cross-fitted sample moment condition⁹⁷

$$\frac{1}{n} \sum_{k=1}^K \sum_{i \in I_k} \psi(W_i; \hat{\theta}_{\times\text{-fit}}, \hat{\eta}_{-k}) = 0. \quad (56)$$

Equation 56 shows that every observation contributes to the estimation of the target parameter, while the nuisance estimate used for that observation is obtained without using any observations from the corresponding held-out fold. The nuisance predictions used in the score are therefore out-of-sample predictions.

This construction reduces overfitting bias by weakening the dependence between the nuisance estimation error $\hat{\eta}_{-k} - \eta_0$ and the observations used to evaluate the score.

⁹⁶Ahrens et al. 2025, p. 20-21.

⁹⁷Ahrens et al. 2025, p. 20-21.

It also avoids permanently reserving one part of the sample for only nuisance estimation or only target-parameter estimation because the roles of the folds are rotated.

In the partially linear regression model, the fold-specific nuisance estimate is

$$\hat{\eta}_{-k} = \left(\hat{\ell}_{-k}, \hat{m}_{-k} \right). \quad (57)$$

For each observation $i \in I_k$, the cross-fitted residuals are

$$\tilde{Y}_i = Y_i - \hat{\ell}_{-k}(X_i), \quad \tilde{D}_i = D_i - \hat{m}_{-k}(X_i). \quad (58)$$

These residuals are based on out-of-sample nuisance predictions because observation i was not used to estimate $\hat{\ell}_{-k}$ or $\hat{m}_{-k}$. After the procedure has been repeated for all folds, every observation has been used once for score evaluation and has contributed to nuisance estimation in the other folds.

Cross-fitting also produces out-of-sample nuisance prediction errors that can be used to assess predictive performance. However, strong predictive performance alone does not guarantee valid DML inference because the required convergence and regularity conditions must also hold.

Standard random-fold cross-fitting is formulated for *independent observations*. When the data are serially dependent, random folds may place closely related observations in the training and evaluation samples. A sample-splitting procedure that respects the temporal dependence structure is therefore required for time-series applications.

5.7 The DML Estimator and Estimation Procedure

When the score function is Neyman orthogonal and the nuisance functions are estimated through cross-fitting, the resulting cross-fitted estimator is the DML estimator. Using the folds and nuisance estimates defined above, the general DML estimator is obtained by solving⁹⁸

$$\frac{1}{n} \sum_{k=1}^K \sum_{i \in I_k} \psi \left(W_i; \hat{\theta}_{\text{DML}}, \hat{\eta}_{-k} \right) = 0. \quad (59)$$

For the partially linear regression model, substituting the cross-fitted residuals from Equation 58 into the orthogonal score gives

$$\frac{1}{n} \sum_{i=1}^n \tilde{D}_i \left(\tilde{Y}_i - \hat{\theta}_{\text{DML}} \tilde{D}_i \right) = 0. \quad (60)$$

Solving Equation 60 for the target parameter gives

$$\hat{\theta}_{\text{DML}} = \frac{\sum_{i=1}^n \tilde{D}_i \tilde{Y}_i}{\sum_{i=1}^n \tilde{D}_i^2}. \quad (61)$$

Equivalently, the estimator can be written directly in terms of the cross-fitted nuisance predictions as

⁹⁸Ahrens et al. 2025, p. 21-23.

$$\hat{\theta}_{\text{DML}} = \frac{\sum_{k=1}^K \sum_{i \in I_k} [D_i - \hat{m}_{-k}(X_i)] [Y_i - \hat{\ell}_{-k}(X_i)]}{\sum_{k=1}^K \sum_{i \in I_k} [D_i - \hat{m}_{-k}(X_i)]^2}. \quad (62)$$

Equation 61 shows that machine learning is first used to estimate the nuisance functions and construct the cross-fitted residuals. The target parameter is then calculated using ordinary least squares (OLS) applied to the residualized outcome and residualized treatment. The numerator is the sum of the products of the two residuals, while the denominator is the sum of the squared treatment residuals. Thus, $\hat{\theta}_{\text{DML}}$ is the OLS slope coefficient from the residual-on-residual regression.

The DML estimation procedure can be summarized in the following steps⁹⁹.

1. **Define the target parameter and score.** The researcher specifies θ_0 , states the assumptions required for its identification, and selects a Neyman-orthogonal score $\psi(W; \theta, \eta)$.
2. **Estimate the nuisance functions.** Within each cross-fitting iteration, machine-learning methods are used to estimate the nuisance functions on the corresponding training sample.
3. **Construct cross-fitted residuals.** The nuisance estimates are evaluated on the held-out observations to obtain $\tilde{Y}_i$ and $\tilde{D}_i$.
4. **Estimate the target parameter.** The residuals from all folds are pooled, and $\hat{\theta}_{\text{DML}}$ is calculated using Equation 61.
5. **Estimate uncertainty and conduct inference.** The cross-fitted score contributions are used to estimate the variance and standard error of the DML estimator.

For a scalar target parameter, define

$$\hat{J} = \frac{1}{n} \sum_{k=1}^K \sum_{i \in I_k} \frac{\partial}{\partial \theta} \psi(W_i; \theta, \hat{\eta}_{-k}) \Big|_{\theta=\hat{\theta}_{\text{DML}}}. \quad (63)$$

The estimated variance of the cross-fitted score contributions is

$$\hat{\Omega} = \frac{1}{n} \sum_{k=1}^K \sum_{i \in I_k} \psi(W_i; \hat{\theta}_{\text{DML}}, \hat{\eta}_{-k})^2. \quad (64)$$

The estimated asymptotic variance and standard error are then

$$\hat{\Sigma} = \hat{J}^{-2} \hat{\Omega}, \quad \widehat{\text{SE}}(\hat{\theta}_{\text{DML}}) = \sqrt{\frac{\hat{\Sigma}}{n}}. \quad (65)$$

The standard error measures the estimated sampling uncertainty of $\hat{\theta}_{\text{DML}}$. It is calculated from the variation in the cross-fitted score contributions and the sensitivity

⁹⁹Ahrens et al. 2025, p. 21-23.

of the score to changes in the target parameter¹⁰⁰.

Under suitable identification, sampling, regularity, and nuisance-convergence conditions, and with a sufficiently large sample, the standardized DML estimator is approximately normally distributed¹⁰¹:

$$\sqrt{n}\hat{\Sigma}^{-1/2}(\hat{\theta}_{\text{DML}} - \theta_0) \xrightarrow{d} \mathcal{N}(0, 1). \quad (66)$$

This limiting distribution refers to the behaviour of the estimator across hypothetical repeated samples. It does not arise from treating the individual cross-fitting folds as separate estimates.

The normal approximation allows a two-sided $(1 - \alpha)$ confidence interval to be constructed as

$$\left[\hat{\theta}_{\text{DML}} - z_{1-\alpha/2} \widehat{\text{SE}}(\hat{\theta}_{\text{DML}}), \hat{\theta}_{\text{DML}} + z_{1-\alpha/2} \widehat{\text{SE}}(\hat{\theta}_{\text{DML}}) \right]. \quad (67)$$

Under repeated sampling and the required assumptions, approximately 95 percent of confidence intervals constructed in this manner would contain the true parameter θ_0 .

Whether the target parameter differs statistically from zero can be evaluated using

$$H_0 : \theta_0 = 0, \quad H_1 : \theta_0 \neq 0. \quad (68)$$

If zero lies outside the confidence interval, the null hypothesis is rejected at the corresponding significance level. If zero lies inside the confidence interval, there is insufficient statistical evidence to reject the null hypothesis.

The variance and standard-error formulas above apply to the standard DML setting with independent observations. In time-series applications, the score contributions may be serially correlated. The variance estimator must therefore be adapted to account for temporal dependence, as discussed in the following section.

Although Neyman orthogonality and cross-fitting reduce regularization and overfitting bias, valid inference still requires sufficiently accurate nuisance estimation, appropriate regularity conditions, and identification of the target parameter. The assumptions required for interpreting θ_0 as a causal effect must therefore be justified independently of the DML estimation procedure¹⁰².

5.8 Interpretation and Limitations

Double Machine Learning should not be interpreted as an automatic solution to causal inference. It provides a framework for estimating a low-dimensional target parameter in the presence of complex or high-dimensional nuisance functions, but it does not replace the need for a valid causal research design. The target parameter must be clearly defined, and the assumptions required for its identification must be justified. In treatment-effect applications, these assumptions may include conditional exogeneity and sufficient treatment variation, while instrumental-variable

¹⁰⁰Ahrens et al. 2025, p. 21-23.

¹⁰¹Ahrens et al. 2025, p. 21-23.

¹⁰²Ahrens et al. 2025.

applications require conditions such as instrument relevance and the exclusion restriction¹⁰³.

In particular, DML cannot correct for omitted or unobserved confounders that are not represented by the included control variables. If important factors jointly affecting the treatment and outcome are missing, the estimated parameter may remain biased even when the nuisance functions are estimated accurately. Consequently, the interpretation of the DML estimate as a causal effect depends on the adequacy of the observed controls and the validity of the underlying identification assumptions.

The quality of nuisance-function estimation also remains important. Neyman orthogonality reduces the first-order effect of nuisance-estimation errors, but it does not allow the nuisance functions to be estimated arbitrarily poorly. Valid inference requires the nuisance estimators to achieve sufficiently accurate convergence and the score function to satisfy the necessary regularity and identification conditions. Poor predictive performance, inappropriate model specification, or insufficient variation in the residualized treatment may therefore weaken the reliability of the final estimate¹⁰⁴.

The empirical results may also be affected by the selected machine-learning algorithm, its hyperparameters, the number and construction of the cross-fitting folds, and the available sample size. In finite samples, different nuisance learners or sample partitions may produce somewhat different estimates. Diagnostic checks and robustness analyses are therefore important for evaluating the stability of the estimated treatment effect. Overall, DML reduces regularization and overfitting bias, but its validity continues to depend on appropriate identification assumptions, sufficiently accurate nuisance estimation, and a cross-fitting procedure that is suitable for the structure of the data.

¹⁰³Chernozhukov et al. 2018.

¹⁰⁴Ahrens et al. 2025.

6 Double Machine Learning for Time-Series Data with Neighbors-Left-Out Cross-Fitting

Standard Double Machine Learning uses cross-fitting to reduce the overfitting bias that may arise when flexible machine-learning methods are used to estimate nuisance functions. Under independent sampling, observations can be randomly divided into several folds. The nuisance functions are estimated using observations outside the evaluation fold, while the observations inside the fold are used to construct the orthogonal score. In this way, an observation is not used both to estimate its own nuisance predictions and to evaluate its own score contribution.

This random cross-fitting procedure is not directly suitable for time-series data. Observations that are close to one another in time may be strongly dependent. Therefore, even when neighboring observations are assigned to different folds, the nuisance-training sample may still contain information that is closely related to the evaluation sample. This weakens the separation between nuisance estimation and target-parameter estimation that cross-fitting is intended to create.

To address this problem, Semenova et al. introduce Neighbors-Left-Out cross-fitting for weakly dependent time-series and dynamic-panel data¹⁰⁵. The central idea is to divide the ordered observations into consecutive blocks. When one block is used for evaluation, the block itself and its immediately neighboring blocks are excluded from nuisance-function estimation. The remaining observations form the quasi-complementary block, referred to below as the quasi-block.

Consider the observed time-series data

$$W_t = (Y_t, D_t, X_t), \quad t = 1, \dots, T, \quad (69)$$

where Y_t denotes the outcome variable, D_t denotes the treatment variable, and X_t denotes a vector of observed control variables. In a time-series application, X_t may also contain lagged or predetermined variables that represent relevant historical information.

The partially linear model is written as

$$Y_t = \theta_0 D_t + g_0(X_t) + U_t, \quad (70)$$

$$D_t = m_0(X_t) + V_t, \quad t = 1, \dots, T, \quad (71)$$

where θ_0 is the target treatment coefficient, $g_0(X_t)$ represents the structural contribution of the control variables to the outcome, and $m_0(X_t)$ represents the conditional treatment function. A causal interpretation of θ_0 requires appropriate identification assumptions, such as

$$\mathbb{E}[U_t \mid D_t, X_t] = 0, \quad (72)$$

$$\mathbb{E}[V_t \mid X_t] = 0. \quad (73)$$

The conditional treatment function is therefore

$$m_0(X_t) = \mathbb{E}[D_t \mid X_t], \quad (74)$$

¹⁰⁵Semenova et al. 2023.

while the conditional outcome function used for partialling out is

$$\ell_0(X_t) = \mathbb{E}[Y_t | X_t]. \quad (75)$$

Taking the conditional expectation of Equation 70 given X_t gives

$$\ell_0(X_t) = \theta_0 m_0(X_t) + g_0(X_t). \quad (76)$$

The nuisance parameter used in the partialling-out procedure is therefore

$$\eta_0 = (\ell_0, m_0). \quad (77)$$

If the true nuisance functions were known, the residualized outcome and treatment would be

$$\tilde{Y}_t = Y_t - \ell_0(X_t), \quad \tilde{D}_t = D_t - m_0(X_t). \quad (78)$$

Using Equation 76, the residualized relationship becomes

$$\tilde{Y}_t = \theta_0 \tilde{D}_t + U_t. \quad (79)$$

The corresponding Neyman-orthogonal score is

$$\psi_{\text{PLR}}(W_t; \theta, \eta) = [Y_t - \ell(X_t) - \theta(D_t - m(X_t))] [D_t - m(X_t)], \quad \eta = (\ell, m). \quad (80)$$

Neyman orthogonality reduces the first-order sensitivity of the score to errors in the estimated nuisance functions. However, the cross-fitting design must also account for the temporal dependence between observations. This is the role of the NLO procedure¹⁰⁶.

To construct the NLO sample splits, the ordered observations are divided into K adjacent and non-overlapping blocks¹⁰⁷,

$$B_1, B_2, \dots, B_K, \quad (81)$$

where each block contains observations from consecutive time periods. The blocks satisfy

$$\bigcup_{k=1}^K B_k = \{W_1, \dots, W_T\}, \quad B_k \cap B_\ell = \emptyset \quad \text{for } k \neq \ell. \quad (82)$$

The chronological order of the observations is preserved. In contrast to standard random cross-fitting, the time-series observations are not randomly shuffled across blocks.

For each evaluation block B_k , define the set containing the block itself and its immediate neighbors as

$$\mathcal{N}(k) = \{k-1, k, k+1\} \cap \{1, \dots, K\}. \quad (83)$$

The quasi-block¹⁰⁸ corresponding to evaluation block B_k is then defined as

¹⁰⁶Semenova et al. 2023.

¹⁰⁷Semenova et al. 2023, p. 481-482.

¹⁰⁸Semenova et al. 2023, p. 481-482.

$$B_k^{\text{qc}} = \bigcup_{\ell \notin \mathcal{N}(k)} B_\ell. \quad (84)$$

The quasi-block B_k^{qc} contains all observations except those belonging to the evaluation block and its immediate neighbors. It is used as the training sample for the nuisance functions associated with evaluation block B_k .

For an interior block B_k , the blocks B_{k-1} , B_k , and B_{k+1} are excluded from nuisance estimation. For the first block, B_1 and B_2 are excluded, while for the final block, B_{K-1} and B_K are excluded. The omitted neighboring blocks create a temporal gap between the training and evaluation observations.

A value of $K \geq 10^{109}$ is suggested so that, when the blocks are approximately equal in size, a sufficiently large proportion of the observations remains available for nuisance-function estimation after the evaluation block and its immediate neighbors have been excluded. In particular, approximately 70 percent or more of the observations can remain in the corresponding quasi-block. This value should be understood as a practical guideline rather than a strict requirement for every application.

For each evaluation block B_k , the conditional outcome and treatment functions are estimated using only the corresponding quasi-block¹¹⁰:

$$\hat{\ell}_k(\cdot) = \hat{\ell}(\cdot; B_k^{\text{qc}}), \quad (85)$$

$$\hat{m}_k(\cdot) = \hat{m}(\cdot; B_k^{\text{qc}}). \quad (86)$$

The fold-specific nuisance estimate is

$$\hat{\eta}_k = (\hat{\ell}_k, \hat{m}_k). \quad (87)$$

Flexible machine-learning methods may be used to estimate these functions, provided that they approximate the conditional outcome and treatment functions sufficiently well.

For every observation $W_t \in B_k$, the NLO cross-fitted residuals are calculated as

$$\tilde{Y}_t = Y_t - \hat{\ell}_k(X_t), \quad \tilde{D}_t = D_t - \hat{m}_k(X_t). \quad (88)$$

The nuisance models used to construct these residuals are estimated without the evaluation block and its neighboring blocks. The residuals are therefore based on predictions obtained from observations that are separated in time from the evaluation observations.

After repeating the procedure for all K blocks, every observation has one NLO cross-fitted outcome residual and one NLO cross-fitted treatment residual. Let $k(t)$ denote the block containing observation W_t . The NLO-DML estimator is obtained by solving

$$\frac{1}{T} \sum_{t=1}^T \psi_{\text{PLR}}(W_t; \hat{\theta}_{\text{NLO}}, \hat{\eta}_{k(t)}) = 0. \quad (89)$$

¹⁰⁹Semenova et al. 2023, p. 483.

¹¹⁰Semenova et al. 2023, p. 483.

Using the cross-fitted residuals, this moment condition can be written as

$$\frac{1}{T} \sum_{t=1}^T \widetilde{D}_t (\widetilde{Y}_t - \hat{\theta}_{\text{NLO}} \widetilde{D}_t) = 0. \quad (90)$$

For the scalar treatment coefficient considered in this thesis, solving Equation 90 gives

$$\hat{\theta}_{\text{NLO}} = \frac{\sum_{t=1}^T \widetilde{D}_t \widetilde{Y}_t}{\sum_{t=1}^T \widetilde{D}_t^2}. \quad (91)$$

Thus, after constructing the NLO cross-fitted residuals, the target coefficient is estimated by applying ordinary least squares to the residualized outcome and residualized treatment. The form of the final estimator is the same as in standard DML. The main difference is that NLO uses a quasi-block that excludes temporally neighboring observations when estimating the nuisance functions.

The theoretical justification for NLO is based on weak temporal dependence. The dependence between an evaluation block B_k and its quasi-block B_k^{qc} can be measured by the beta-mixing coefficient¹¹¹

$$\gamma(B_k, B_k^{\text{qc}}) = \frac{1}{2} \|P_{B_k, B_k^{\text{qc}}} - P_{B_k} \times P_{B_k^{\text{qc}}}\|_{\text{TV}}, \quad (92)$$

where $P_{B_k, B_k^{\text{qc}}}$ denotes the joint distribution of the two blocks, P_{B_k} and $P_{B_k^{\text{qc}}}$ denote their marginal distributions, and $\|\cdot\|_{\text{TV}}$ denotes total variation distance. If the two blocks are independent, their joint distribution equals the product of their marginal distributions and the dependence coefficient is zero.

Leaving out the neighboring blocks increases the temporal distance between the evaluation block and the observations used for nuisance estimation. Under beta-mixing, dependence becomes weaker as this distance increases¹¹². The NLO construction therefore makes the evaluation block and its quasi-block approximately independent when the temporal dependence decreases sufficiently quickly.

The formal argument is based on Strassen’s coupling result¹¹³. It allows the dependent evaluation block and quasi-block to be compared with independent blocks having the same marginal distributions. The approximation error is controlled by the dependence coefficient in Equation 92. NLO does not make the original observations exactly independent, but under the required weak-dependence conditions, the independent-block approximation becomes increasingly accurate.

Although NLO cross-fitting reduces the dependence between the observations used for nuisance estimation and score evaluation, it does not necessarily eliminate serial correlation among the resulting cross-fitted score contributions. For inference, the score contributions are retained in their original chronological order, and a

¹¹¹Semenova et al. 2023, p. 484.

¹¹²Semenova et al. 2023, pp. 482–484.

¹¹³Semenova et al. 2023, pp. 482–484.

heteroskedasticity- and autocorrelation-consistent (HAC) variance estimator is applied to account for both heteroskedasticity and remaining serial dependence in the score process¹¹⁴. The empirical HAC specification, including the selected maximum lag, is reported in the empirical application.

The NLO procedure therefore preserves the main logic of Double Machine Learning while adapting cross-fitting to the ordered structure of time-series data. Neyman orthogonality reduces the first-order effect of nuisance-estimation errors, while the quasi-block construction reduces the dependence between nuisance-function estimation and score evaluation. HAC estimation subsequently accounts for remaining serial dependence in the score contributions when conducting statistical inference.

¹¹⁴Müller [2014](#).

7 Empirical Application

7.1 Dataset Description

The empirical analysis in this thesis is based on the **Open Power System Data (OPSD) Time Series** data package, version 2020-10-06¹¹⁵. The dataset provides time-series observations relevant for electricity-system modelling, including electricity consumption, load forecasts, day-ahead electricity prices, actual wind and solar electricity generation, and installed renewable generation capacities. The data are collected and processed from official sources, particularly European transmission system operators and the ENTSO-E Transparency Platform. They are aggregated by country, control area, and electricity bidding zone and cover 32 European countries. Detailed information about the dataset, its variables, and its processing procedure is available at [Open Power System Data – Time Series](#).

This thesis uses the `time_series_60min_singleindex.csv` file, in which each observation represents the beginning of a one-hour period. The dataset contains both the Coordinated Universal Time variable `utc_timestamp` and the corresponding Central European local-time variable `cet_cest_timestamp`. Where the original observations are available at a higher temporal resolution, such as 15-minute or 30-minute intervals, OPSD aggregates them into hourly averages for the 60-minute dataset. The complete data package mainly covers the period from 2015 to 2020.

Due to the broad geographical scope and the large number of variables contained in the original dataset, the empirical analysis is restricted to variables related to the German electricity system. Germany provides a particularly relevant case study because its electricity system is characterised by a substantial contribution from variable renewable energy sources, especially wind and solar power. The weather-dependent nature of these technologies causes considerable variation in electricity generation across hours, days, and seasons. The German data therefore provide a suitable empirical setting for examining the relationship between renewable electricity generation and day-ahead electricity prices using machine-learning and causal-inference methods.

The following paragraphs introduce the main variables used in the empirical analysis and explain their economic meanings:

Electricity demand is primarily represented by `DE_load_actual_entsoe_transparency`, which measures the total realised electricity load in Germany in megawatts (MW). This variable reflects the total electricity consumed by households, industry, commercial activities, and other users connected to the electricity system. The variable `DE_load_forecast_entsoe_transparency` contains the corresponding day-ahead load forecast and represents the expected level of electricity demand before the delivery period. Both actual and forecast load are relevant because day-ahead electricity prices are determined on the basis of expected market conditions, while differences between expected and realised demand provide additional information about electricity-system dynamics.

Renewable electricity supply is represented by `DE_solar_generation_actual` and

¹¹⁵Energieökonomik 2020.

DE_wind_generation_actual. The first variable measures actual hourly electricity generation from photovoltaic installations in Germany, while the second measures total actual wind generation, including electricity production from both onshore and offshore wind installations. Both variables are expressed in MW. Solar generation follows a strong daily cycle because it is generally zero during the night and reaches higher levels during daylight hours. Wind generation does not follow the same regular intraday pattern but can fluctuate substantially across consecutive hours because of changing weather conditions.

The variables **DE_solar_capacity** and **DE_wind_capacity** measure the installed generation capacity of solar and wind technologies in Germany, respectively. These variables are also expressed in MW. In contrast to actual generation, installed capacity changes relatively gradually and captures the structural expansion of renewable electricity technologies during the observation period. Including these capacity variables helps distinguish short-term fluctuations in actual renewable generation from longer-term developments in the structure of the German electricity system.

The outcome variable of the empirical analysis is **DE_LU_price_day_ahead**, which represents the hourly day-ahead spot price for the Germany–Luxembourg bidding zone. The day-ahead price is determined before the physical delivery of electricity and reflects market participants’ expectations regarding the balance between expected electricity demand and available electricity supply. In the empirical analysis, this variable is measured in euros per megawatt-hour (EUR/MWh). Although the variable formally represents the joint Germany–Luxembourg bidding zone, this study uses it as the relevant day-ahead electricity price measure for Germany and does not analyse Luxembourg separately.

7.2 Data Preprocessing

The original **time_series_60min_singleindex.csv** file contains 50,401 hourly observations and 300 variables covering electricity-system data for several European countries and regions. Since this thesis focuses on the German electricity system, the original dataset was reduced to variables relevant to electricity demand, renewable electricity generation, installed renewable-generation capacity, and the day-ahead electricity price. Data preprocessing and feature engineering were conducted in Python. The **pandas** library was primarily used for data manipulation and time-series processing, while the **NumPy** library was used to construct the cyclical calendar variables.

First, the variable **utc_timestamp** was converted from a character string to a timezone-aware datetime format. The corresponding local timestamp, **cet_cest_timestamp**, was subsequently generated using the **Europe/Berlin** timezone. This conversion accounts for the distinction between Central European Time and Central European Summer Time. Based on the local timestamp, two additional calendar variables were created. The variable **hour** identifies the hour of the day, while **dayofweek** identifies the day of the week using values from 0 to 6, where Monday is represented by 0 and Sunday by 6.

Nine variables were initially retained from the original dataset. These comprised the two timestamps, actual and forecast electricity load, actual solar and wind gener-

ation, installed solar and wind capacity, and the Germany–Luxembourg day-ahead electricity price. Together with the two calendar variables constructed from the local timestamp, the dataset contained 11 variables at this stage of the preprocessing procedure.

The selected dataset initially contained all 50,401 hourly observations. However, the target variable, `DE_LU_price_day_ahead`, was available for only 17,540 observations. Observations outside the period for which the target variable was reported were therefore excluded from the empirical sample.

The remaining observations covered the period from 30 September 2018 at 23:00 UTC to 30 September 2020 at 22:00 UTC. A continuity check was conducted by comparing the observed UTC timestamps with a complete hourly time index generated using `pandas.date_range`. The check identified four missing hourly timestamps and no duplicated UTC timestamps. These four observations had initially been excluded because their day-ahead electricity prices were missing, although the remaining electricity-system variables were available.

To preserve the continuous hourly structure of the time series, the four corresponding rows were recovered from the initially selected dataset and reintroduced into the empirical sample. After this procedure, the dataset contained 17,544 consecutive hourly observations without missing or duplicated UTC timestamps.

Missing values in the installed-capacity variables were treated separately from missing values in the outcome variable. Missing values in `DE_solar_capacity` and `DE_wind_capacity` were completed using forward filling, followed by backward filling where no earlier observation was available. This approach was considered appropriate because installed generation capacity generally changes gradually rather than fluctuating from one hour to the next. The use of backward filling was restricted to observations for which no preceding capacity value was available. The four missing values of `DE_LU_price_day_ahead` were replaced using linear interpolation between the neighbouring hourly price observations.

Before constructing the final modelling dataset, the temporal dependence of the day-ahead electricity price was examined using the autocorrelation function (ACF). As shown in Figure 5, the electricity-price series exhibits strong positive autocorrelation. Prices observed in consecutive hours are therefore closely related, although the correlation gradually decreases as the lag length increases. Local increases in the autocorrelation coefficients around the 24-hour and 48-hour lags additionally indicate a recurring daily pattern in the electricity-price series. These findings suggest that previous price observations and recurring calendar patterns contain relevant information for modelling the current day-ahead electricity price.

This temporal structure is particularly important for the subsequent XGBoost analysis. XGBoost operates on a tabular feature matrix and does not automatically incorporate the chronological ordering of time-series observations into the model structure¹¹⁶. Additional variables were therefore constructed to represent the temporal characteristics of the electricity-price series in a form that could be used as model inputs.

¹¹⁶Chen and Guestrin 2016.

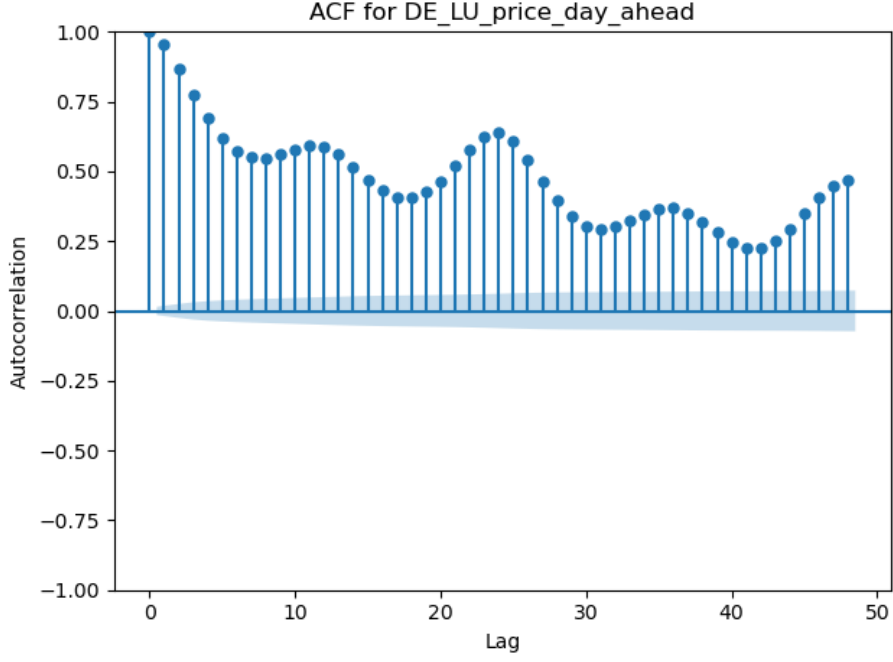


Figure 5: Autocorrelation function of the day-ahead electricity price.

First, the variables `hour` and `dayofweek` were constructed from the local timestamp to represent recurring intraday and weekly patterns. Since hours and weekdays follow cyclical rather than strictly linear structures, sine and cosine transformations were created for both variables. These transformations allow the model to recognise that observations at the beginning and end of a cycle are close to each other. For example, hour 23 is immediately followed by hour 0, even though their original numerical values are far apart¹¹⁷.

Lagged electricity-price variables were subsequently created to capture the strong temporal dependence identified by the ACF analysis. These variables contain the electricity price observed one hour, 24 hours, and 168 hours before the current observation. The one-hour lag captures short-term price persistence, whereas the 24-hour and 168-hour lags represent recurring price patterns at the same hour of the preceding day and the preceding week, respectively. Such lagged-variable construction transforms information from previous time-series observations into input features that can be processed by supervised machine-learning models¹¹⁸.

In addition, rolling price features were constructed to summarise recent market conditions. The 24-hour rolling mean represents the average price level during the preceding day, while the 24-hour rolling standard deviation measures price variability over the same period. These variables provide the model with information about both the recent price level and the degree of short-term market volatility¹¹⁹.

All lagged and rolling variables were calculated exclusively from price information observed before the current period. More specifically, the price series was shifted by

¹¹⁷Chakraborty and Elzarka 2019.

¹¹⁸Chakraborty and Elzarka 2019.

¹¹⁹Băra and Oprea 2025.

one hour before the rolling statistics were calculated. Consequently, the day-ahead electricity price at time t was not used to construct its own explanatory features. This procedure prevents direct target leakage and preserves the chronological ordering of the time-series data.

In total, 11 additional variables were created during the preprocessing and feature-engineering stages. These consisted of two basic calendar variables, four cyclical calendar variables, three lagged price variables, and two rolling price variables. Since the 168-hour lag requires a complete price history of one week, the first 168 observations contained incomplete feature information and were excluded from the final modelling dataset.

The final preprocessed dataset, denoted by `df_final`, contains 17,376 hourly observations and 20 columns. These columns comprise two timestamp variables, the outcome variable, and 17 explanatory variables.

7.3 Exploratory Data Analysis

Before conducting the predictive and causal analyses, the main variables were examined descriptively to identify their temporal patterns, seasonal variation, and relationships over the observation period. The following figures provide an initial overview of the day-ahead electricity price, electricity demand, and renewable electricity generation:

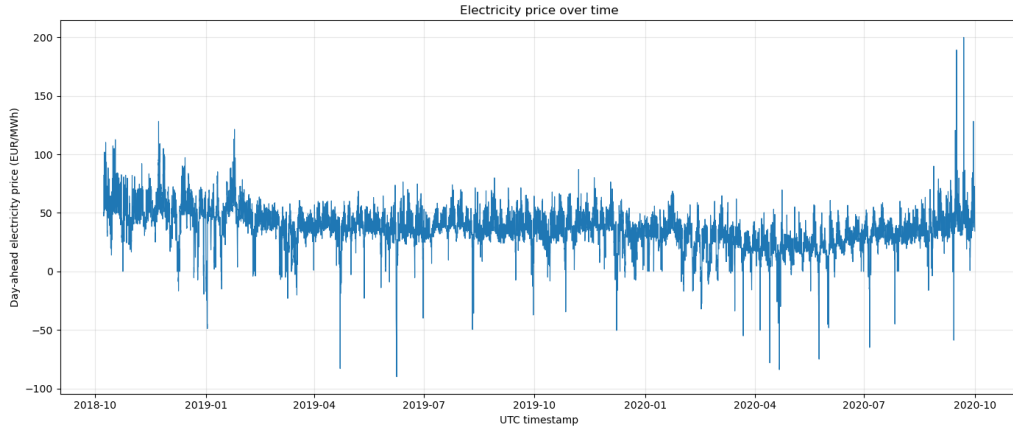


Figure 6: Hourly day-ahead electricity prices over the observation period.

Figure 6 presents the hourly day-ahead electricity price for the Germany–Luxembourg bidding zone. The series exhibits considerable short-term volatility, including several pronounced positive and negative price spikes. Although prices remain positive during most hours, negative values occur at several points in the sample and do not necessarily represent data errors. Negative electricity prices may occur when available electricity supply exceeds demand. Such situations can be associated with high electricity generation from wind or solar technologies, low electricity demand, limited storage capacity, and the technical inflexibility of some conventional power plants. Under these conditions, electricity producers may accept negative market prices rather than reduce or temporarily discontinue generation. The negative price observations were therefore retained because they represent actual electricity-market behaviour and may contain relevant information for the subsequent empirical analysis.

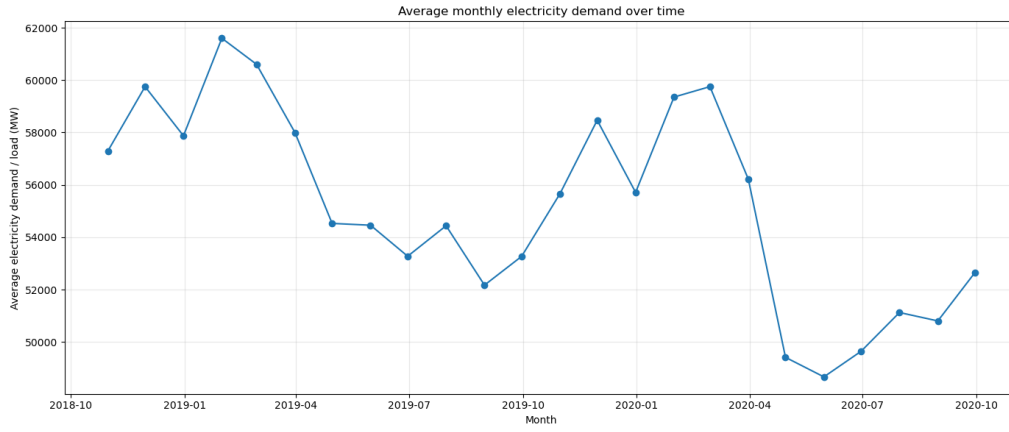


Figure 7: Average monthly electricity demand over the observation period.

Figure 7 shows the development of average monthly electricity demand. Demand displays a clear seasonal pattern, with higher average levels during the winter months and lower levels during the summer months. The highest monthly values are observed during the winter of 2018–2019, when average electricity demand exceeds approximately 60,000 MW. A further increase can be observed during the winter of 2019–2020, followed by a pronounced decline in spring 2020. Average demand remains comparatively low during the subsequent summer months before increasing again toward the end of the observation period. These variations indicate that calendar and seasonal information may be relevant for modelling electricity demand and prices.

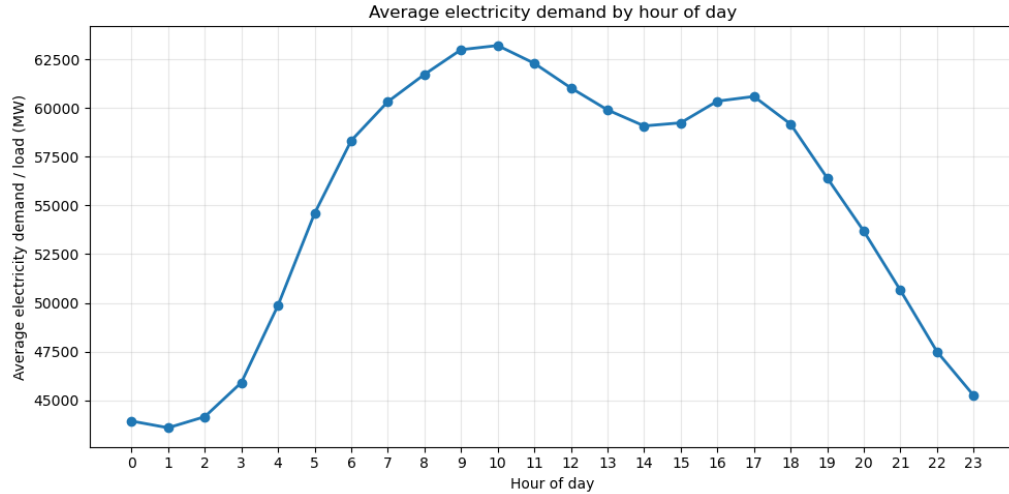


Figure 8: Average electricity demand by hour of the day.

The intraday pattern of electricity demand is presented in Figure 8. Average demand is lowest during the night, particularly between midnight and 03:00. It begins to increase rapidly in the early morning and reaches its highest level at approximately 09:00–10:00. After the morning peak, demand decreases moderately during the afternoon before showing a smaller secondary increase at approximately 16:00–17:00. It then declines continuously throughout the evening. This pattern is consistent with daily household, commercial, and industrial activity and demonstrates the importance of controlling for the hour of the day in the subsequent empirical models. Figure 9 compares actual electricity demand with the combined actual generation from solar and wind technologies. Electricity demand is typically higher than the

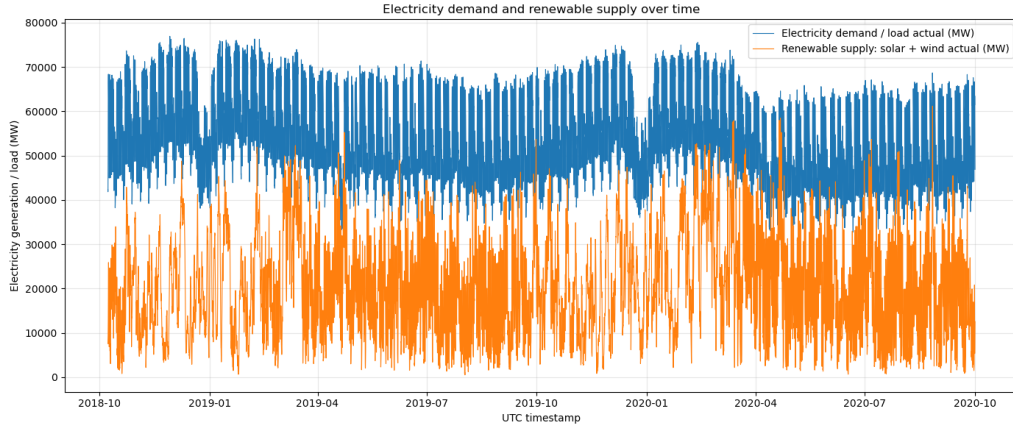


Figure 9: Electricity demand and combined solar and wind generation over time.

electricity generated by these two renewable sources. However, the size of the difference varies considerably over time because renewable generation is strongly affected by weather conditions. During periods of high renewable generation, the difference between demand and combined solar and wind generation becomes substantially smaller. During periods of low renewable generation, a larger share of electricity demand must be met by other generation technologies, electricity storage, or net electricity imports. The figure therefore illustrates the time-varying contribution of solar and wind technologies to the German electricity system.

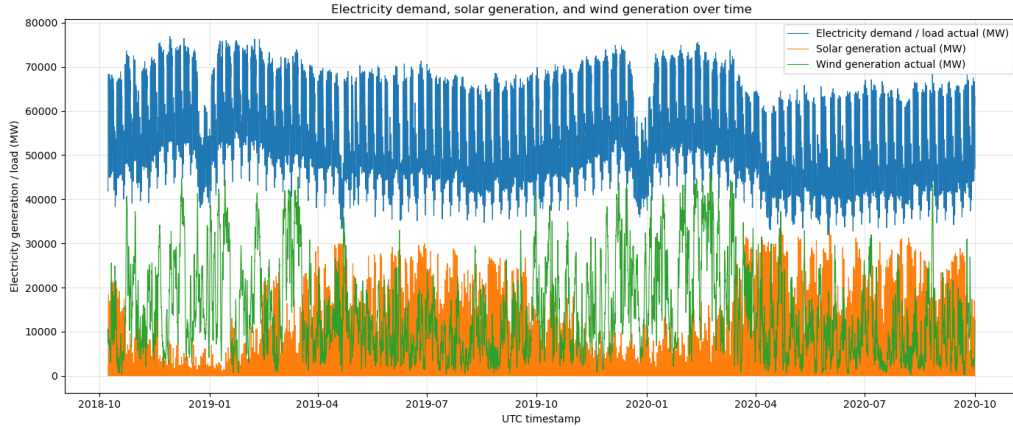


Figure 10: Electricity demand, solar generation, and wind generation over time.

Figure 10 presents solar and wind generation separately. The two renewable technologies exhibit substantially different temporal patterns. Solar generation follows a pronounced daily and seasonal cycle. It is zero or close to zero during nighttime hours and is considerably higher during the spring and summer months than during winter.

Wind generation, in contrast, does not follow the same regular intraday cycle. It is highly variable and contains periods of both very high and very low generation throughout the year. Wind generation nevertheless tends to be higher during several winter and early-spring periods. These differences indicate that solar and wind generation should be included separately in the empirical analysis rather than represented by a single aggregated renewable generation variable.

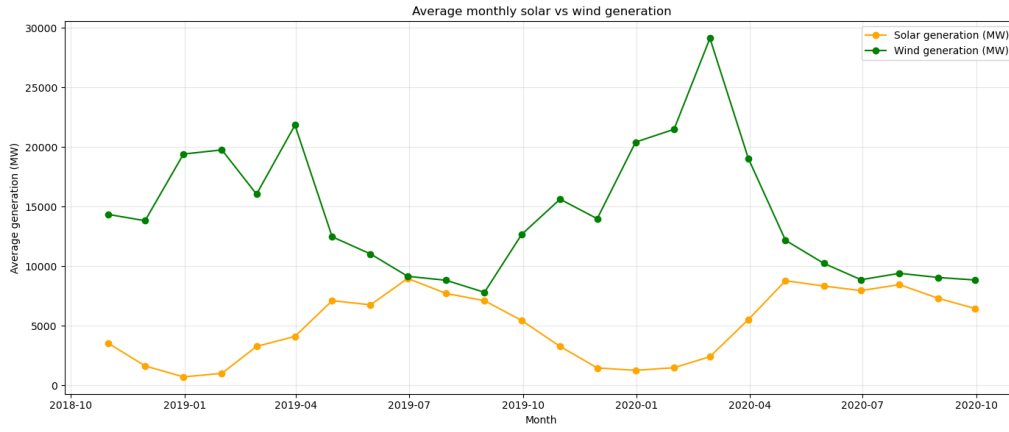
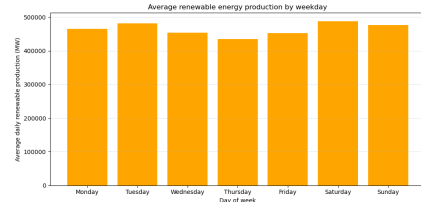
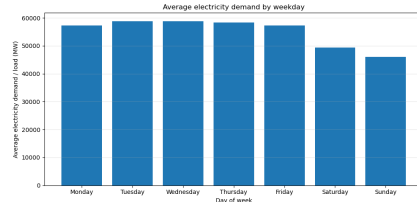


Figure 11: Average monthly solar and wind electricity generation.

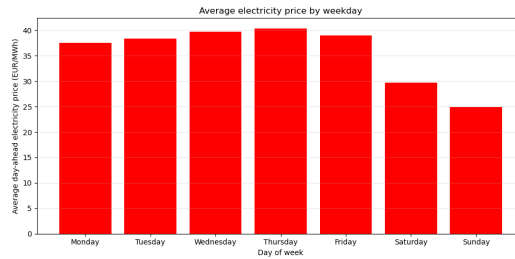
The different seasonal patterns are more clearly visible in Figure 11. Average solar generation is lowest during the winter months and increases substantially during spring and summer. In contrast, average wind generation is comparatively high during several winter and early-spring months but varies considerably from one month to another. Average wind generation exceeds average solar generation during most months in the observation period. However, the difference becomes smaller during summer, when solar generation reaches its highest average levels and wind generation is comparatively lower. The figure therefore suggests that wind generation contributed a larger average quantity of electricity than solar generation over the sample period, while solar generation displayed a more regular seasonal cycle.



(a) Average renewable generation by weekday.



(b) Average electricity demand by weekday.



(c) Average day-ahead electricity price by weekday.

Figure 12: Average renewable generation, electricity demand, and day-ahead electricity price by weekday.

Figure 12 compares average renewable generation, electricity demand, and day-ahead electricity prices across the days of the week. Renewable generation varies only moderately across weekdays and remains comparatively high during the weekend. Electricity demand, by contrast, is clearly lower on Saturday and Sunday than

on working days.

The average day-ahead electricity price follows a similar weekly pattern. Prices are highest from Wednesday to Friday and decline substantially during the weekend. In particular, Sunday exhibits both the lowest average demand and the lowest average electricity price, while renewable generation remains relatively high.

7.4 Empirical Results

7.4.1 XGBoost Model Specification and Predictive Performance

This section presents the empirical results of the electricity-price analysis. The analysis is divided into three stages. First, the predictive performance of the XGBoost model is evaluated using the coefficient of determination and the root mean squared error calculated on the held-out test set. Second, SHAP values are used to examine how the individual explanatory variables contribute to the XGBoost predictions. Finally, Neighbors-Left-Out Double Machine Learning (NLO-DML) is applied to estimate the effects of actual solar and wind generation on the German day-ahead electricity price.

The XGBoost regression model was specified with 350 boosting iterations, a learning rate of 0.03, and a maximum tree depth of four. The minimum child weight was set to five to restrict the creation of terminal nodes based on very small groups of observations. In addition, both the row-subsampling and column-subsampling rates were set to 0.80 to reduce the risk of overfitting. The regularization parameters were set to zero for the L_1 penalty and one for the L_2 penalty.

The predictive performance of the model was evaluated on the held-out test set using the coefficient of determination, R^2 , and the root mean squared error, RMSE. The resulting performance measures are summarized in Table 1.

Table 1: Predictive performance of the XGBoost model on the test set.

Performance measure	Test-set result
R^2	0.9172
RMSE	4.7732 EUR/MWh

The test-set R^2 of 0.9172 indicates that the XGBoost model explains approximately 91.72% of the observed variation in the German day-ahead electricity price within the test sample. The RMSE of 4.7732 EUR/MWh indicates that the predictions deviate from the observed prices by approximately 4.77 EUR/MWh in root-mean-square terms. Overall, these values indicate strong out-of-sample predictive performance.

Figure 13 compares the actual and predicted electricity prices over the test period. The predicted series follows the overall movement of the observed price series relatively closely. In particular, the model captures regular short-term fluctuations, negative-price periods, and most medium-sized price movements. The substantial



Figure 13: Actual and predicted German day-ahead electricity prices in the test set.

overlap between the two series is consistent with the high test-set R^2 .

Nevertheless, the predicted series appears smoother than the actual series during several extreme price episodes. Some unusually high positive price peaks are underestimated by the model. This suggests that XGBoost represents regular electricity-price dynamics well but has greater difficulty predicting rare and unusually large price movements. Such events occur less frequently in the dataset and may be driven by exceptional market conditions that are not fully represented by the included explanatory variables.

7.4.2 SHAP-Based Interpretation of the XGBoost Model

SHAP values were calculated to examine how the input variables contribute to the electricity-price predictions generated by the XGBoost model. Two complementary SHAP plots are considered. The mean absolute SHAP plot presents the average magnitude of each variable’s contribution, whereas the SHAP summary plot additionally shows the direction and distribution of these contributions across the observations.

Figure 14 shows that the electricity price lagged by one hour, `price_lag_1h`, is by far the most influential predictor in the XGBoost model. Its mean absolute SHAP value is approximately 9.53, which is substantially larger than the importance values of the remaining variables. This result is consistent with the strong short-term autocorrelation previously observed in Figure 5. The electricity price in the preceding hour therefore contains substantial predictive information about the current price.

Actual solar and wind generation are the second- and third-most important predictors, with mean absolute SHAP values of approximately 1.26 and 1.23, respectively. Their similar importance indicates that both renewable-generation variables contribute meaningfully to the prediction of German day-ahead electricity prices.

Temporal variables also contribute to the model predictions. The cyclical variable `hour_sin` has a mean absolute SHAP value of approximately 1.10, indicating that

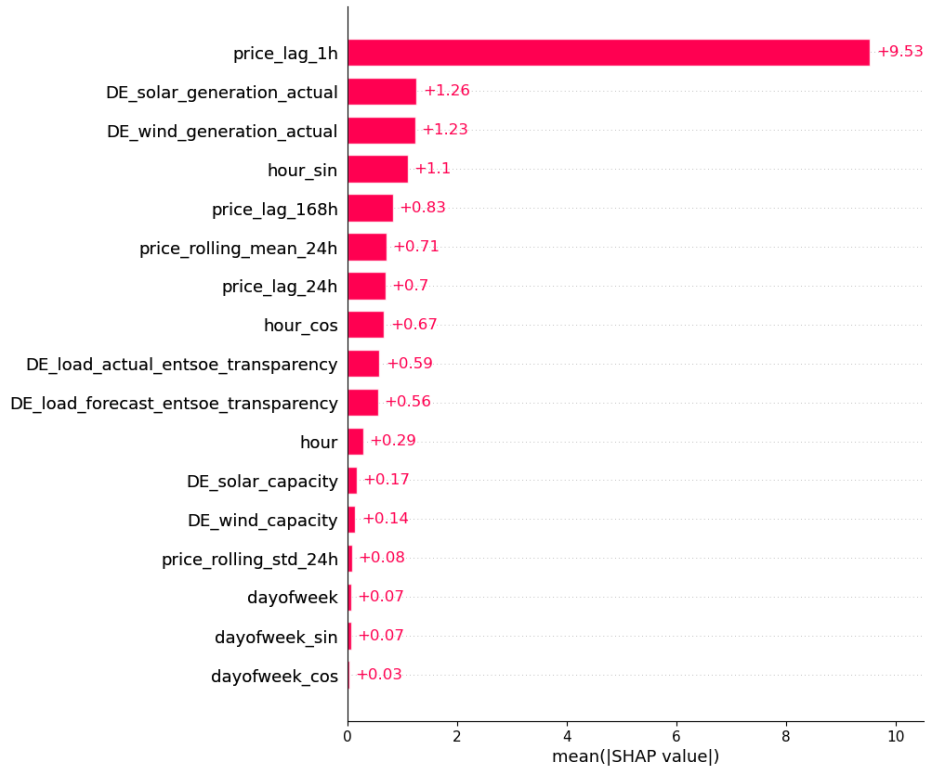


Figure 14: Global feature importance measured by mean absolute SHAP values.

systematic intraday price patterns are important. The weekly price lag, `price_lag_168h`, has a mean absolute SHAP value of approximately 0.83, while the 24-hour rolling mean and the daily price lag have values of approximately 0.71 and 0.70, respectively. These findings show that the model uses a combination of short-term persistence and recurring daily and weekly price patterns.

Actual and forecast electricity demand also contribute to the predictions, with mean absolute SHAP values of approximately 0.59 and 0.56, respectively. By contrast, installed solar and wind capacity have substantially lower average importance than actual generation. This result is plausible because installed capacity changes only gradually over time, whereas actual generation varies from hour to hour according to weather conditions and operational availability.

Figure 15 provides information about both the magnitude and the direction of the feature contributions. Each point represents one observation. Its horizontal position indicates the corresponding SHAP value, while its colour represents the observed value of the feature. Positive SHAP values increase the model prediction relative to its baseline value, whereas negative SHAP values reduce the prediction.

A clear directional pattern is visible for `price_lag_1h`. High values of the previous-hour price, represented by red observations, are generally associated with positive SHAP values and therefore increase the predicted current price. Conversely, low values of the lagged price are mainly associated with negative SHAP values and reduce the predicted price. This pattern is consistent with the strong persistence observed in the hourly electricity-price series.

High values of actual solar generation are predominantly located on the negative

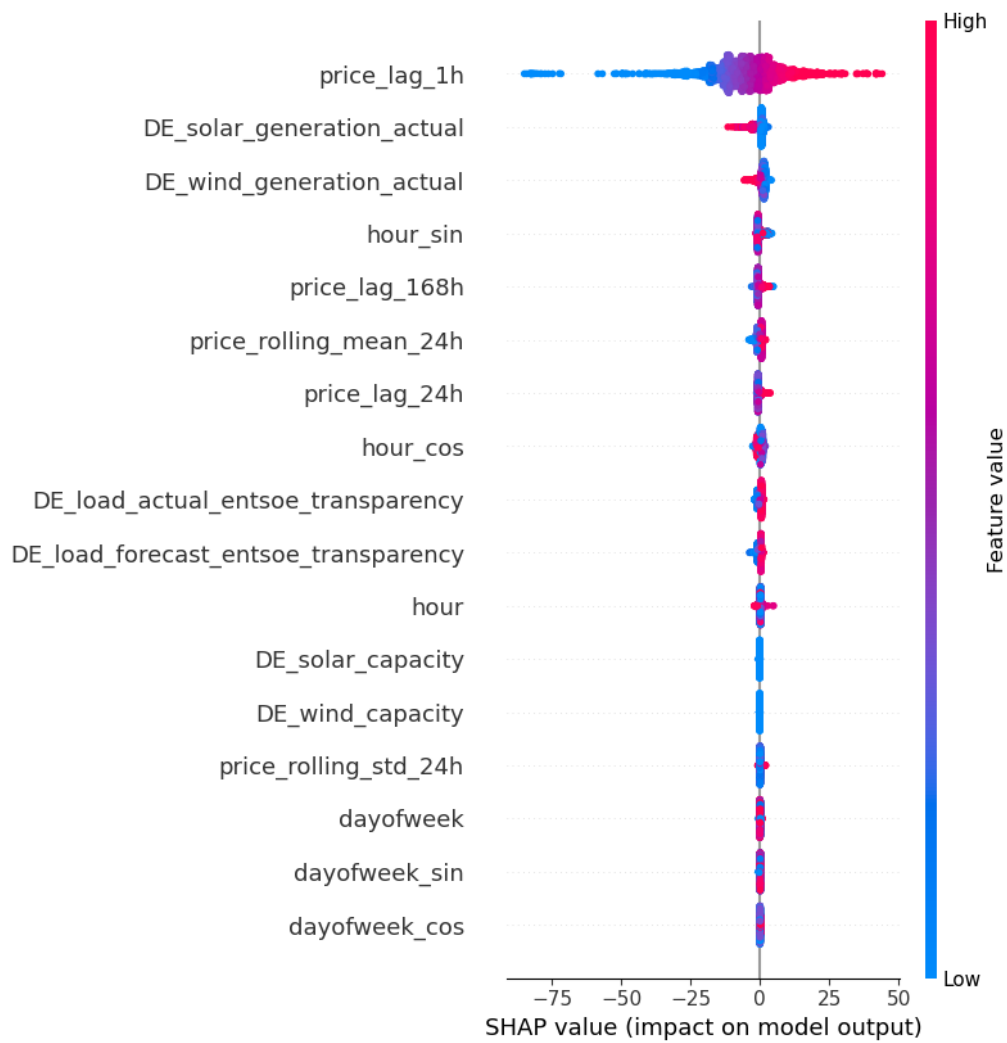


Figure 15: SHAP summary plot showing the magnitude and direction of the feature contributions to the XGBoost predictions.

side of the SHAP axis. This indicates that high solar generation generally reduces the electricity-price prediction produced by the XGBoost model. A similar pattern is observed for actual wind generation. High wind-generation values tend to produce negative SHAP contributions, whereas lower wind-generation values tend to have neutral or moderately positive contributions. The fitted prediction model therefore associates high renewable generation with lower German day-ahead electricity prices.

The SHAP results must, however, be distinguished from the subsequent causal analysis. SHAP values describe how the fitted XGBoost model uses each variable when generating predictions. They measure predictive contributions and do not establish that an intervention that changes a particular variable would causally change the electricity price. Consequently, the negative SHAP contributions of solar and wind generation should be interpreted as predictive relationships rather than causal effects. The causal interpretation is examined separately using NLO-DML.

7.4.3 NLO-DML Estimate for Solar Generation

The first NLO-DML specification treats actual solar generation as the treatment variable and the German day-ahead electricity price as the outcome. The nuisance models estimate the parts of the outcome and treatment that can be explained by the included control variables. The final-stage coefficient is subsequently estimated from the relationship between the cross-fitted residualized electricity price and residualized solar generation.

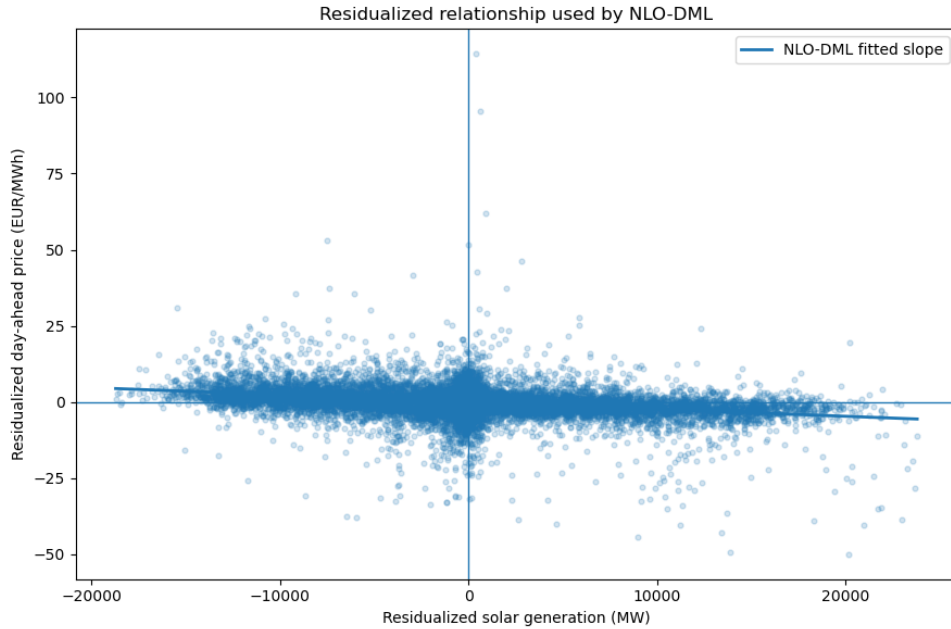


Figure 16: Residualized German day-ahead electricity price plotted against residualized solar generation.

The estimated coefficient for actual solar generation is

$$\hat{\theta}_{\text{solar}} = -0.000235 \text{ EUR/MWh per additional MW.} \quad (93)$$

Because the coefficient per MW is numerically small, it is more informative to express the estimate per GW of generation:

$$\hat{\theta}_{\text{solar}} = -0.2351 \text{ EUR/MWh per additional GW.} \quad (94)$$

The estimate indicates that, after partialling out the variation explained by the included control variables, an additional 1 GW of solar generation is associated with an average reduction of approximately 0.2351 EUR/MWh in the German day-ahead electricity price.

The heteroskedasticity- and autocorrelation-consistent (HAC) standard error is 0.0180 EUR/MWh per GW. The corresponding 95% confidence interval is

$$\text{CI : } [-0.2704, -0.1999] \text{ EUR/MWh per GW.} \quad (95)$$

Since the entire confidence interval lies below zero, the null hypothesis of no solar effect is rejected at the 5% significance level. The reported p -value of approximately 4.69×10^{-39} provides strong statistical evidence that the estimated coefficient differs from zero.

Figure 16 shows the residualized relationship used in the final stage of the NLO-DML estimation. The fitted line has a negative slope, which is consistent with the sign of the estimated coefficient. The slope appears visually shallow because solar generation is measured in MW and the horizontal axis covers a range of several tens of thousands of MW.

A large proportion of the observations is concentrated around zero residualized solar generation. Several large residualized price observations nevertheless remain, indicating that some exceptional price movements are not fully explained by the included controls and nuisance models. The use of HAC standard errors is therefore appropriate because the residualized score process may remain both heteroskedastic and serially correlated.

Under the maintained identification assumptions of the partially linear DML model, including the assumption that the included control variables sufficiently account for the relevant confounding structure, the negative coefficient can be interpreted as an estimated causal effect. Economically, the result is consistent with the merit-order mechanism: additional solar generation with low marginal operating costs may displace more expensive conventional generation from the market-clearing process.

7.4.4 NLO-DML Estimate for Wind Generation

The second NLO-DML specification treats actual wind generation as the treatment variable while retaining the German day-ahead electricity price as the outcome. As in the solar specification, the final coefficient is estimated from the relationship between the cross-fitted residualized outcome and the residualized treatment.

The estimated coefficient for actual wind generation is

$$\hat{\theta}_{\text{wind}} = -0.000102 \text{ EUR/MWh per additional MW.} \quad (96)$$

Expressed per GW, the coefficient is

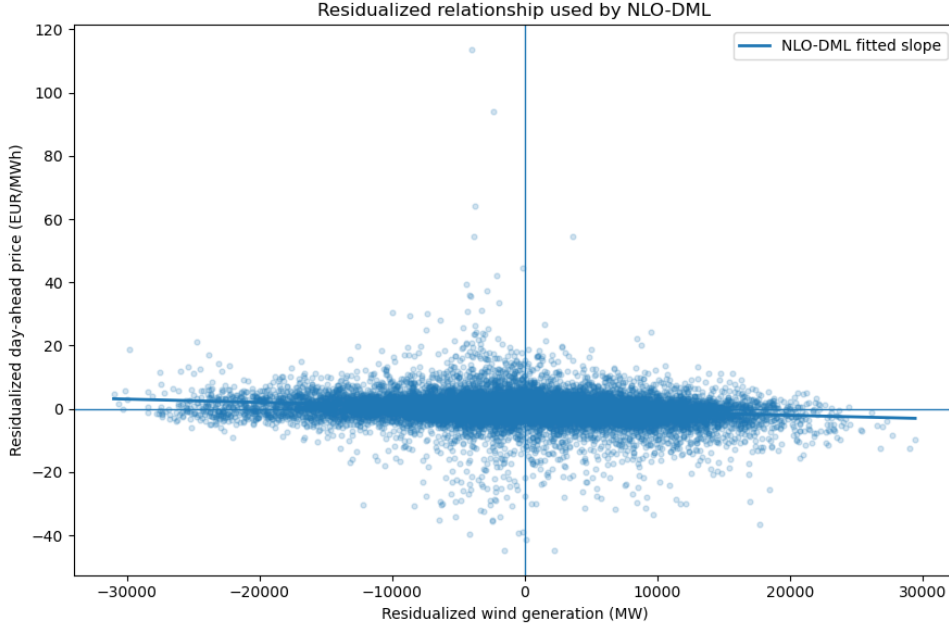


Figure 17: Residualized German day-ahead electricity price plotted against residualized wind generation.

$$\hat{\theta}_{\text{wind}} = -0.1022 \text{ EUR/MWh per additional GW.} \quad (97)$$

The estimate indicates that an additional 1 GW of wind generation is associated with an average reduction of approximately 0.1022 EUR/MWh in the German day-ahead electricity price, after accounting for the variation explained by the included control variables.

The HAC standard error is 0.0098 EUR/MWh per GW, and the corresponding 95% confidence interval is

$$\text{CI : } [-0.1215, -0.0830] \text{ EUR/MWh per GW.} \quad (98)$$

The confidence interval lies entirely below zero. In addition, the reported p -value is approximately 2.46×10^{-25} . The null hypothesis of no wind effect is therefore rejected at the 5% significance level.

The residualized scatter plot in Figure 17 displays a negative fitted slope, which is consistent with the estimated coefficient. As in the solar specification, the slope appears relatively small because wind generation is measured in MW. A large number of observations are concentrated close to zero residualized wind generation, while several positive and negative residualized price outliers remain.

The dispersion of the residualized price observations also differs across the range of residualized wind generation. This pattern supports the use of HAC standard errors, which account for potential heteroskedasticity and serial dependence in the time-series score process.

Subject to the maintained identification assumptions, the negative coefficient may be interpreted as evidence of a price-reducing causal effect of wind generation. As with solar generation, this finding is consistent with the merit-order mechanism,

according to which additional electricity from low-marginal-cost renewable sources may displace more expensive conventional generation from the market-clearing process.

8 Discussion

The predictive and causal analyses provide related but conceptually different findings. The XGBoost model achieves strong out-of-sample predictive performance, with an R^2 of 0.9172 and an RMSE of 4.7732 EUR/MWh. The model therefore captures most of the observed variation in the German day-ahead electricity price. However, the predicted series is smoother than the actual series and tends to underestimate some extreme positive price movements. This suggests that the included variables represent regular electricity-price dynamics well, while exceptional market situations remain more difficult to predict.

The SHAP analysis shows that the electricity price lagged by one hour is the most important predictor. This finding is consistent with the strong short-term persistence of hourly electricity prices. Actual solar and wind generation are also among the most important non-price predictors. High levels of solar and wind generation generally produce negative SHAP values, indicating that the XGBoost model associates higher renewable generation with lower electricity-price predictions. However, these SHAP results describe predictive relationships and should not be interpreted as causal effects.

The NLO-DML analysis goes beyond the predictive interpretation by estimating the effects of solar and wind generation after accounting for the variation explained by the included control variables. Table 2 summarizes the estimated effects.

Table 2: NLO-DML estimates of the effects of renewable generation on the day-ahead electricity price.

Treatment	Estimate per GW	HAC standard error	95% confidence interval
Solar generation	−0.2351	0.0180	[−0.2704, −0.1999]
Wind generation	−0.1022	0.0098	[−0.1215, −0.0830]

Both coefficients are negative and statistically significant. Under the maintained identification assumptions, an additional 1 GW of solar generation is associated with an average reduction of approximately 0.2351 EUR/MWh in the German day-ahead electricity price. An additional 1 GW of wind generation is associated with an average reduction of approximately 0.1022 EUR/MWh. These findings are consistent with the merit-order mechanism, according to which electricity generated by technologies with low marginal operating costs can replace more expensive conventional generation in the market-clearing process.

The estimated solar coefficient is approximately 2.3 times as large in absolute magnitude as the estimated wind coefficient. One possible explanation is that solar generation is concentrated during daylight hours, when electricity demand and prices may be relatively high. Additional solar generation during these hours may therefore replace comparatively expensive marginal generation. Wind generation is distributed less regularly across the day and depends on different weather conditions, which may partly explain its smaller estimated average effect.

Nevertheless, the solar and wind coefficients were estimated in separate treatment-

specific NLO-DML models. Their magnitudes can therefore be compared descriptively, but the results do not provide a formal statistical test that the solar effect is larger than the wind effect. Moreover, the estimated effects on the day-ahead electricity price alone are not sufficient to determine which technology is preferable for future investment. Such a conclusion would also depend on weather conditions, generation availability, investment costs, installed capacity, grid requirements, and other energy-system considerations. Overall, the results indicate that both solar and wind generation are promising contributors to reducing wholesale electricity prices.

9 Extensions and Future Research

This thesis estimated the effects of solar and wind generation using two separate NLO-DML specifications. Although the estimated solar effect was larger in absolute magnitude, the current results do not provide a formal statistical test of whether solar generation has a stronger price effect than wind generation. A natural extension would therefore be to include both renewable-generation variables within a joint causal model and directly test the hypothesis

$$H_0 : \theta_{\text{solar}} = \theta_{\text{wind}}. \quad (99)$$

Such an analysis would make it possible to determine whether the difference between the estimated effects is statistically significant rather than only descriptively different. It would also account more directly for the relationship between solar and wind generation within the same electricity system.

A further extension would compare the estimated price effects with the costs of expanding the corresponding renewable-generation technologies. This comparison should consider the construction and installation costs of photovoltaic and wind-generation facilities, as well as grid-connection costs, maintenance costs, generation availability, and expected operating life. Combining these cost measures with the estimated causal price effects could provide a broader assessment of which technology produces a greater electricity-price reduction relative to the required investment. However, such an extension would require additional time and research effort and therefore lies beyond the scope of the present thesis.

Another possible methodological extension is provided by the recent study *Double Machine Learning for Time Series*, published in 2026, which introduces a new methodology called Reverse Cross-Fitting for the application of DML to time-series data¹²⁰. Future research could examine the theoretical assumptions and estimation procedure of this newly proposed method and investigate whether it is suitable for hourly electricity-market data. If the required conditions are satisfied, Reverse Cross-Fitting could be applied to extend the time-series DML analysis conducted in this thesis.

¹²⁰Ciganovic, D’Amario, and Tancioni 2026.

10 Conclusion

The thesis examined the role of solar and wind generation in the German day-ahead electricity market by combining predictive modelling, model explainability, and causal inference. XGBoost was used to predict electricity prices, SHAP was applied to explain the resulting model predictions, and Double Machine Learning with Neighbors-Left-Out cross-fitting was used to estimate the effects of renewable generation in an hourly time-series setting.

The predictive analysis showed that electricity prices can be modelled accurately using historical prices, renewable generation, electricity demand, and temporal variables. The SHAP results further indicated that short-term price persistence is the main source of predictive information, while solar and wind generation are also important predictors and are generally associated with lower predicted electricity prices. However, these predictive relationships were kept conceptually separate from causal interpretation.

Under the maintained identification assumptions, the NLO-DML results indicate that both solar and wind generation have price-reducing effects on German day-ahead electricity prices. These findings are consistent with the merit-order mechanism and suggest that renewable electricity can contribute not only to reducing the environmental impact of conventional generation but also to lowering wholesale electricity prices. Nevertheless, the results do not determine which renewable technology is generally preferable, since such a conclusion would also require consideration of investment costs, weather conditions, generation availability, storage, and grid requirements.

Overall, the thesis demonstrates the importance of distinguishing prediction, model explanation, and causal inference. XGBoost and SHAP provide insights into electricity-price prediction, while NLO-DML offers a framework for estimating causal effects in temporally dependent data. The study therefore contributes to a more complete understanding of how renewable generation is related to electricity prices and provides a methodological basis for further research on causal effects in energy-market time series.

Literature

- [1] Achim Ahrens et al. “An introduction to double/debiased machine learning”. In: *arXiv preprint arXiv:2504.08324* (2025).
- [2] Naomi Altman and Martin Krzywinski. “Points of Significance: Association, correlation and causation.” In: *Nature methods* 12.10 (2015).
- [3] Adela Bâra and Simona-Vasilica Oprea. “Transformer-based forecasting with synthetic input data generation for day-ahead electricity markets”. In: *Journal of King Saud University Computer and Information Sciences* 37.8 (2025), p. 233.
- [4] Alexandre Belloni, Victor Chernozhukov, and Christian Hansen. “High-dimensional methods and inference on structural and treatment effects”. In: *Journal of Economic Perspectives* 28.2 (2014), pp. 29–50.
- [5] Candice Bentéjac, Anna Csörgő, and Gonzalo Martínez-Muñoz. “A comparative analysis of gradient boosting algorithms”. In: *Artificial Intelligence Review* 54.3 (2021), pp. 1937–1967.
- [6] Debaditya Chakraborty and Hazem Elzarka. “Advanced machine learning techniques for building performance simulation: a comparative analysis”. In: *Journal of Building Performance Simulation* 12.2 (2019), pp. 193–207.
- [7] Tianqi Chen and Carlos Guestrin. “Xgboost: A scalable tree boosting system”. In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016, pp. 785–794.
- [8] Victor Chernozhukov et al. *Double/debiased machine learning for treatment and structural parameters*. 2018.
- [9] Suman Chowdhury and Dilip Kumar Das. “Renewable Energy Trends in Future Prospect Analyzed by XGBoost-A Time Series Analysis”. In: *2024 4th International Conference on Sustainable Expert Systems (ICSES)*. IEEE. 2024, pp. 134–138.
- [10] Milos Ciganovic, Federico D’Amario, and Massimiliano Tancioni. “Double machine learning for time series”. In: *The Econometrics Journal* (2026), utag019.
- [11] I Covert and SI Lee. “Improving kernelshap: Practical shapley value estimation via linear regression. arXiv 2020”. In: *arXiv preprint arXiv:2012.01536* ().
- [12] Scott Cunningham. *Causal inference: The mixtape*. Yale university press, 2021.
- [13] Barry De Ville. “Decision trees”. In: *Wiley Interdisciplinary Reviews: Computational Statistics* 5.6 (2013), pp. 448–455.
- [14] Neon Neue Energieökonomik. *Load, wind and solar, prices in hourly resolution*. 2020. URL: https://data.open-power-system-data.org/time_series/2020-10-06.
- [15] Leonida Gianfagna and Antonio Di Cecco. *Explainable AI with python*. Vol. 10. Springer, 2021.
- [16] Jason S Haukoos and Roger J Lewis. “The propensity score”. In: *Jama* 314.15 (2015), pp. 1637–1638.
- [17] Md. Mirajul Islam. *Decision Tree* — mirajul1995. <https://medium.com/@mirajul1995/decision-tree-a47ae3f8a987>. [Accessed 30-07-2026].

- [18] Uday Kamath and John Liu. *Explainable artificial intelligence: an introduction to interpretable machine learning*. Vol. 2. Springer, 2021.
- [19] Wei-Yin Loh. “Classification and regression trees”. In: *Wiley interdisciplinary reviews: data mining and knowledge discovery* 1.1 (2011), pp. 14–23.
- [20] Fernando López. *Ensemble Learning: Bagging, Boosting*. 2017. URL: <https://towardsdatascience.com/ensemble-learning-bagging-boosting-3098079e5422/> (visited on 06/10/2026).
- [21] Haiping Lu and Shuo Zhou. *Turing Course: An Introduction to Transparent Machine Learning*. 2022. URL: <https://alan-turing-institute.github.io/Intro-to-transparent-ML-course/index.html> (visited on 02/08/2025).
- [22] Scott M Lundberg, Gabriel Erion, et al. “From local explanations to global understanding with explainable AI for trees”. In: *Nature machine intelligence* 2.1 (2020), pp. 56–67.
- [23] Scott M Lundberg and Su-In Lee. “A unified approach to interpreting model predictions”. In: *Advances in neural information processing systems* 30 (2017).
- [24] Ulrich K Müller. “HAC corrections for strongly autocorrelated time series”. In: *Journal of Business & Economic Statistics* 32.3 (2014), pp. 311–322.
- [25] Ashley I Naimi and Brian W Whitcomb. “Defining and identifying average treatment effects”. In: *American Journal of Epidemiology* 192.5 (2023), pp. 685–687.
- [26] Alexey Natekin and Alois Knoll. “Gradient boosting machines, a tutorial”. In: *Frontiers in neurorobotics* 7 (2013), p. 63623.
- [27] Woon Yee Ng et al. “Causal SHAP: Feature Attribution with Dependency Awareness through Causal Discovery”. In: *2025 International Joint Conference on Neural Networks (IJCNN)*. IEEE. 2025, pp. 1–8.
- [28] Judea Pearl, Madelyn Glymour, and Nicholas P. Jewell. *Causal Inference in Statistics: A Primer*. John Wiley & Sons, 2016.
- [29] Ana Victoria Ponce-Bobadilla et al. “Practical guide to SHAP analysis: Explaining supervised machine learning model predictions in drug development”. In: *Clinical and translational science* 17.11 (2024), e70056.
- [30] Butch Quinto. “Introduction to machine learning”. In: *Next-Generation Machine Learning with Spark: Covers XGBoost, LightGBM, Spark NLP, Distributed Deep Learning with Keras, and More*. Springer, 2020, pp. 1–27.
- [31] Chava L Ramspek et al. “Prediction or causality? A scoping review of their conflation within current observational research”. In: *European journal of epidemiology* 36.9 (2021), pp. 889–898.
- [32] Paul R Rosenbaum and Donald B Rubin. “The central role of the propensity score in observational studies for causal effects”. In: *Biometrika* 70.1 (1983), pp. 41–55.
- [33] Omer Sagi and Lior Rokach. “Ensemble learning: A survey”. In: *Wiley interdisciplinary reviews: data mining and knowledge discovery* 8.4 (2018), e1249.
- [34] Vira Semenova et al. “Inference on heterogeneous treatment effects in high-dimensional dynamic panels under weak dependence”. In: *Quantitative Economics* 14.2 (2023), pp. 471–510.

- [35] Lloyd S Shapley et al. “A value for n-person games”. In: (1953).
- [36] Galit Shmueli. “To explain or to predict?” In: *Statistical science* (2010), pp. 289–310.
- [37] Jilei Yang. “Fast treeshap: Accelerating shap value computation for trees”. In: *arXiv preprint arXiv:2109.09847* (2021).

List of Figures

1	Basic structure of a decision tree. ¹²¹	4
2	Bagging and Boosting. ¹²²	5
3	Confounding structure in a DAG.	20
4	Collider structure in a DAG.	21
5	Autocorrelation function of the day-ahead electricity price.	42
6	Hourly day-ahead electricity prices over the observation period.	43
7	Average monthly electricity demand over the observation period.	44
8	Average electricity demand by hour of the day.	44
9	Electricity demand and combined solar and wind generation over time.	45
10	Electricity demand, solar generation, and wind generation over time.	45
11	Average monthly solar and wind electricity generation.	46
12	Average renewable generation, electricity demand, and day-ahead electricity price by weekday.	46
13	Actual and predicted German day-ahead electricity prices in the test set.	48
14	Global feature importance measured by mean absolute SHAP values.	49
15	SHAP summary plot showing the magnitude and direction of the feature contributions to the XGBoost predictions.	50
16	Residualized German day-ahead electricity price plotted against residualized solar generation.	51
17	Residualized German day-ahead electricity price plotted against residualized wind generation.	53

List of Abbreviations

ACF	– Autocorrelation Function
ANN	– Artificial Neural Network
ATE	– Average Treatment Effect
ATTE	– Average Treatment Effect on the Treated
ATTU	– Average Treatment Effect on the Untreated
CI	– Confidence Interval
DAG	– Directed Acyclic Graph
DeepLIFT	– Deep Learning Important Features
DML	– Double Machine Learning
ENTSO-E	– European Network of Transmission System Operators for Electricity
GW	– Gigawatt
HAC	– Heteroskedasticity- and Autocorrelation-Consistent
LIME	– Local Interpretable Model-Agnostic Explanations
ML	– Machine Learning
MSE	– Mean Squared Error
MW	– Megawatt
MWh	– Megawatt-hour
NLO	– Neighbors-Left-Out
NLO-DML	– Neighbors-Left-Out Double Machine Learning
OLS	– Ordinary Least Squares
OPSD	– Open Power System Data
PLR	– Partially Linear Regression
ReLU	– Rectified Linear Unit
RMSE	– Root Mean Squared Error
SE	– Standard Error
SHAP	– SHapley Additive exPlanations
UTC	– Coordinated Universal Time
XAI	– Explainable Artificial Intelligence
XGBoost	– Extreme Gradient Boosting

Academic integrity declaration

I hereby declare that I have written all parts of this thesis independently and that I have not used any sources other than those indicated in the thesis and that I have not submitted the thesis in the same or a similar form for any other examination. All verbatim or analogous copies and quotations have been identified and verified.

Information on the use of AI-based tools

I affirm that I have not used any AI-based tools whose use has been explicitly excluded in writing by the examiner. I am aware that the use of texts or other content and products generated by AI-based tools does not guarantee their quality. I am fully responsible for the adoption of any machine-generated passages used by me and bear the responsibility for any erroneous or distorted content, faulty references, violations of data protection and copyright law or plagiarism generated by the AI. I also affirm that my creative influence predominates in the present work.

Berlin, 03.08.2026

(Place, Date)

A handwritten signature in blue ink, appearing to read 'Eing', is written over a horizontal line.

(Signature)